Mizan

Phase 1

CS 340 Introduction To Database Systems

Dr.

Section:

Norah Alfayez - 222410215

Reema alkheraif - 222410580

Mashael aljarbou - 221410453

Jana Altalhi - 221410589

Contents

Description of the Team

Our team is a group of four dedicated students working together on this project. We have chosen to make an application, "Mizan," as a helping tool for managing money. Our team will combine the various skills and ideas to see that this project is ultimately a success. We believe in teamwork and good communication so that we can complete our work on time and in quality.

Introduction

Our project is a personal finance managing application called "Mizan," ("ميزان"). Often, people lose track of their own spending and hence face problems in saving. Mizan will help them in tracking every income and expense.

The application would facilitate users to record all their financial transactions; itemize them into categories that might include "food" or "shopping"; and produce cool reports and charts that illustrate against their income and expenses, highlighting exactly where their money goes. The users could even set their monthly budgets and get notified by the app whenever they exceed their expenses.

In this phase, we shall explain the aim of the application, what it will do, what data must be stored in the database, and how it is going to look. We shall also demonstrate how we plan to work together as a set of four.

Purpose and Scope of the Application

What is the application for?

The application tries to be a simple yet powerful personal money management tool. It replaces paper notebooks and overly complicated spreadsheets. Gaining quick insight into one's spending habits aids an individual in controlling overspending.

Who will use this application?

From whoever handles money, those would find this application useful. Great for students to meet their allowance needs; it allows working people to track how they spend their salaries; families use it to plan the household budget.

Main Goals

In brief, the major things the application is supposed to implement are as follows:

- Allow users to create a private secure account.
- Allow a user to add any new income (salary, gifts, investment returns).

- Allow a user to add new expenses (food, rent, and coffee).
- Allow the user to organize their spending into custom categories.
- Allow the user to see a list of all transactions ever.
- Allow the user to set monthly budgets for either total spending or spending for a category.
- Show the user the fastest summary of financial status for the month via the dashboard.
- Give the user visual charts and reports in the explanation of spending patterns.
- Send smart notifications to warn the user when they are close to exceeding their budget.

Data Requirements

In order to carry out all these features, implementation requires populating a database with a variety of information. So far, we have identified core entities.

List of Data Do We Need to Store

1. User Information
2. Transaction Information (income and expenses recorded)
3. Category Information (Transaction groups)
4. Budget Information (Spending limits given by the user)

Description of Each Data Item

User

We store all the information about the people using the application in this table.

- **User ID**: a sequential global number attributed to every user (Primary Key).
- **Username**: the name that the user uses on registration.
- **Email**: the user's email address, it serves the system for authentication.
- **Password**: an encrypted version of the user entered password and stored as a hashed form.
- **Phone Number**: the user's mobile number (optional field).
- **Created At**: Monitored system unique date stamps when a user registers.

Transaction

This acts as the fundamental table. It captures every unique transaction financially.

- **Transaction ID**: every transaction identifier (Primary Key) is unique for that single transaction.
- **User ID**: Shows the link between the transaction and the user who created it. (Foreign Key to User table).

- **Amount**: the sum of money for the transaction (for instance, 150.75 SAR).
- **Type**: specifies whether it is an **Income** or an **Expense**.
- **Category ID**: This refers to the category of this record (e.g., Food, Salary). (Foreign Key to Category table).
- **Date**: The date the transaction occurred.
- **Description**: A short note written by the user (e.g., "Lunch at AlBaik Food", "Monthly Salary").
- **Payment Method**: How the payment was made (e.g., Cash, Credit Card, Apple Pay).

Category

This table stores the different groups for organizing transactions.

- **Category ID**: A unique number for each category (Primary Key).
- **User ID**: Connects to the user who created this category. Some default categories will have a system User ID.
- **Name**: The name of the category (e.g., "Rent", "Entertainment", "Grocery").
- **Type**: Indicates if this category is for Income or Expense.
- **Icon**: A name for a small icon image to make the interface user-friendly (e.g., "food-icon.png").

Budget

This table holds the spending limits that users set for themselves.

- **Budget ID**: A unique number for each budget setting (Primary Key).
- **User ID**: Connects this budget to the user who created it (Foreign Key to User table).
- **Category ID**: (Optional) Links to a specific category if this budget is for a category. If `NULL`, it is a general monthly budget.
- **Amount**: The maximum amount of money the user allows you to spend.
- **Start Date**: The start date for the budget period (e.g., first day of the month).
- **End Date**: The end date for the budget period (e.g., last day of the month).
- **Is Active**: A status (Yes/No) to turn the budget on or off.

Functional and Non-Functional Requirements:

Functional Requirements

User Account Management

- The system shall allow a new user to register by providing a unique email, a username, and a password.
- The system shall allow a registered user to log in using their correct email and password.

- The system shall prevent login if the email is not registered or the password is incorrect.
- The system shall provide a "Forgot Password" feature that sends a password reset link to the user's email.

Transaction Management

- The system shall allow a logged-in user to add a new income transaction, requiring an amount, category, and date.
- The system shall allow a logged-in user to add a new expense transaction, requiring an amount, category, and date.
- The system shall allow a user to edit the details (amount, date, category, description) of their existing transactions.
- The system shall allow a user to delete their existing transactions.
- The system shall display a paginated list of all transactions for the logged-in user.
- The system shall allow users to filter the transaction list by date range, category, type (income or expense), and payment method.

Category Management

- The system shall provide a set of default categories (such as Food, Transportation, Salary) for all new users.
- The system shall allow a user to create new custom categories.
- The system shall allow a user to edit or delete only the custom categories they created.

Budget Management

- The system shall allow a user to set a general monthly budget for total expenses.
- The system shall allow a user to set a budget for a specific expense category for a month.
- The dashboard shall visually display the user's current spending compared to their active budgets, such as a progress bar.

Dashboard and Reporting

- The system shall display a dashboard upon login showing:
    - ❖ Total Income for the current month.
    - ❖ Total Expenses for the current month.
    - ❖ The remaining balance (Income minus Expenses).
- A notification for any budget that is getting close to its limit or has been exceeded.
- The system shall generate a pie chart showing the distribution of expenses by category for a user-selected period.
- The system shall generate a line chart showing the trend of income versus expenses over the last six months.

Notifications

- The system shall display a warning notification on the dashboard when spending for a budget reaches 80% of the budgeted amount.
- The system shall display a critical alert notification on the dashboard when a budget is exceeded.

Non-Functional Requirements

Usability (Easy to Use)

- The user interface shall be easy to understand. A new user should be able to add their first transaction without reading a manual.
- The process to add a transaction shall take three clicks or less from the dashboard.

Performance (Speed)

- The dashboard shall load and display all summary data within three seconds for a user with up to 1,000 transactions.
- Applying a filter to the transactions list shall show results in under two seconds.

Security (Keeping Data Safe)

- User passwords shall be stored with a strong, modern hashing algorithm (like bcrypt).
- Users shall only be able to access, view, modify, or delete their own data. Data between users must be completely separate.
- All data transmission between the client and server shall be encrypted using HTTPS.

Reliability (Always Working)

- The application shall have an uptime of 99%.
- The database shall be designed to prevent data loss in case of a system failure.

Compatibility (Works Everywhere)

- The web application shall work properly on the latest versions of Chrome, Firefox, Safari, and Edge.

Paper Prototype of the User Interface

Explanation of the Prototype

We have sketched the main screens of the Mizan application. This allows us to visualize the user experience and agree on the design before we start coding. The prototype shows the key workflows.

Pictures of the Paper Screens

Screen 1

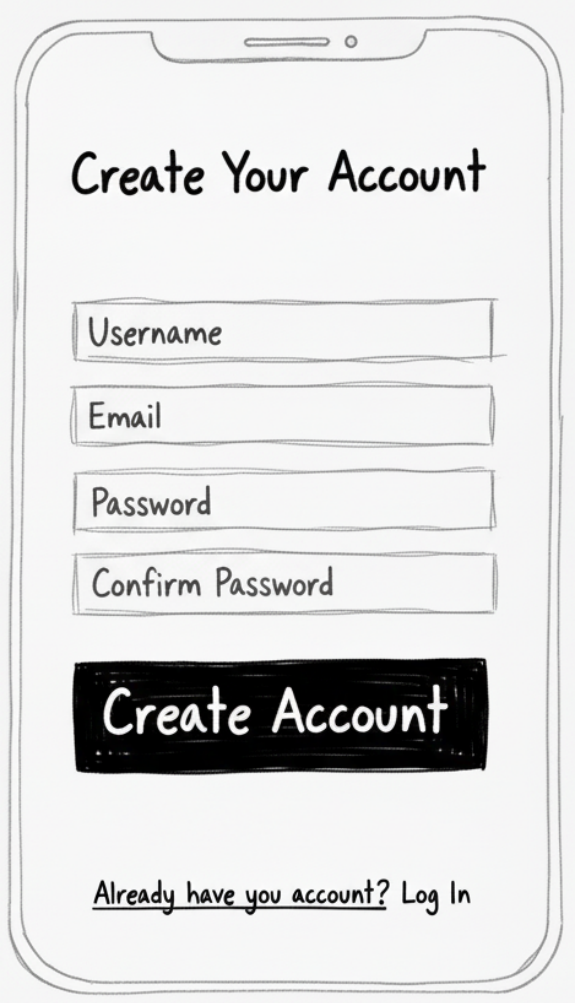
- Welcome: This is the very first screen a new user sees. It explains what the app does and encourages them to start.

Screen 2

- **Login**: This is the screen that a non-logged in user will use to log in.

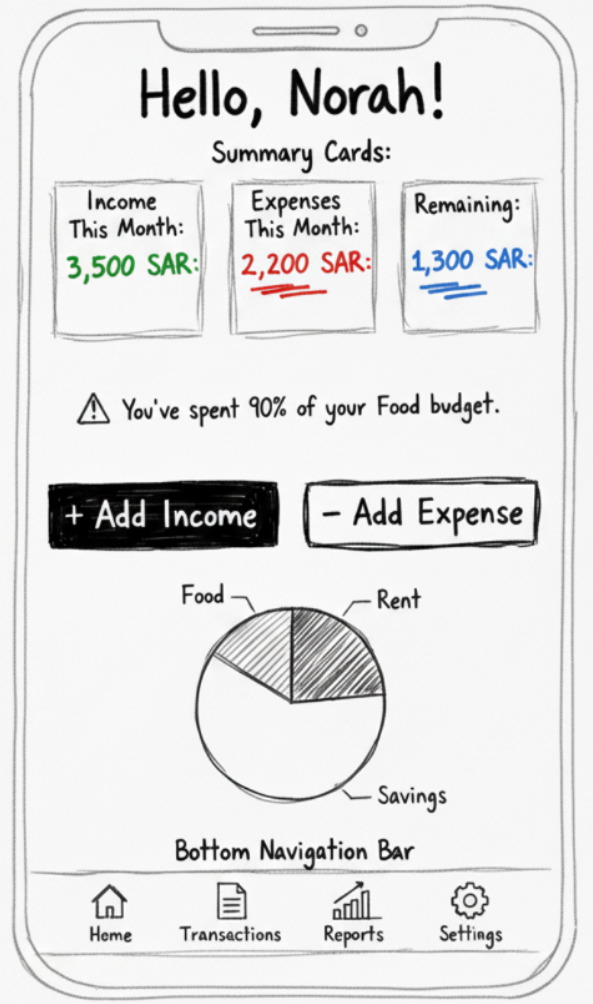
Screen 3

- **Registration Screen**: this is where a new user creates an account.



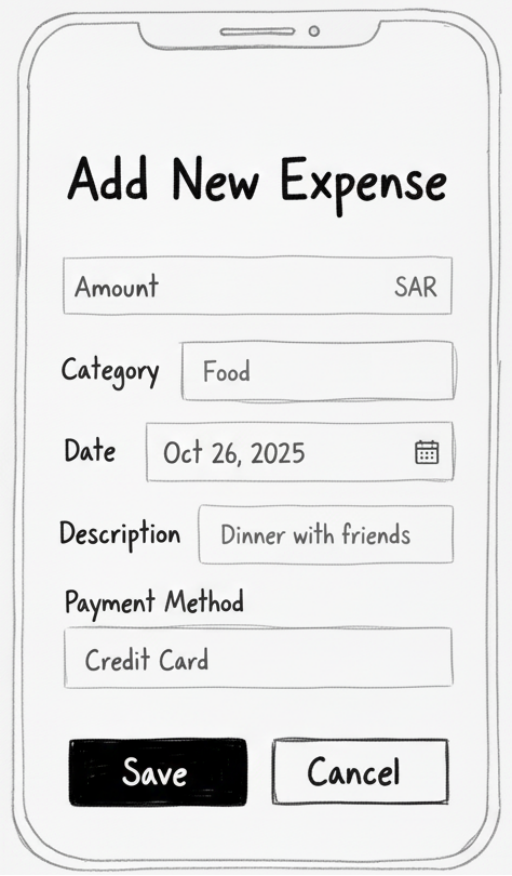
Screen 4

- **Main Dashboard**: The main hub of the app. It gives the user a quick snapshot of their financial health.



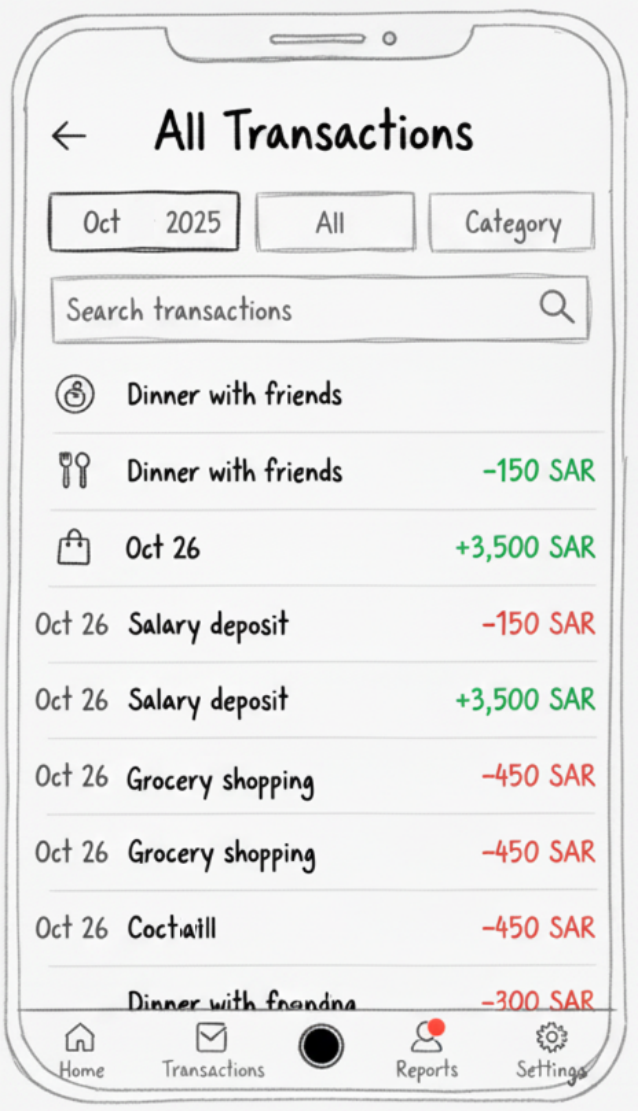
Screen 5

- **'Add Expense' Form**: The form a user fills out to add new income or an expense. The title will be changed between “Add New Expense” and “Add new Income”



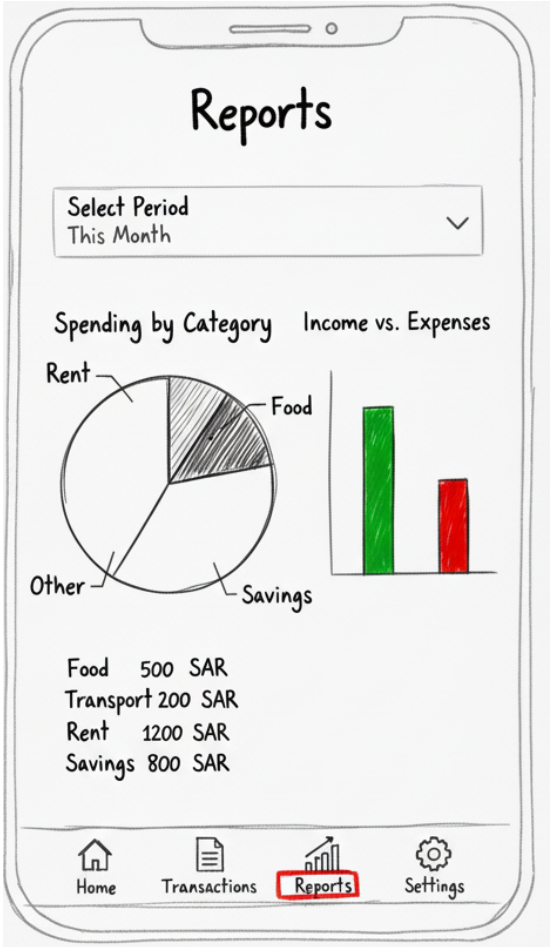
Screen 6

- **Transactions List Page**: Shows the user a history of all their transactions.



Screen 7

- **Reports Page**: Shows the user visual charts about their spending habits.

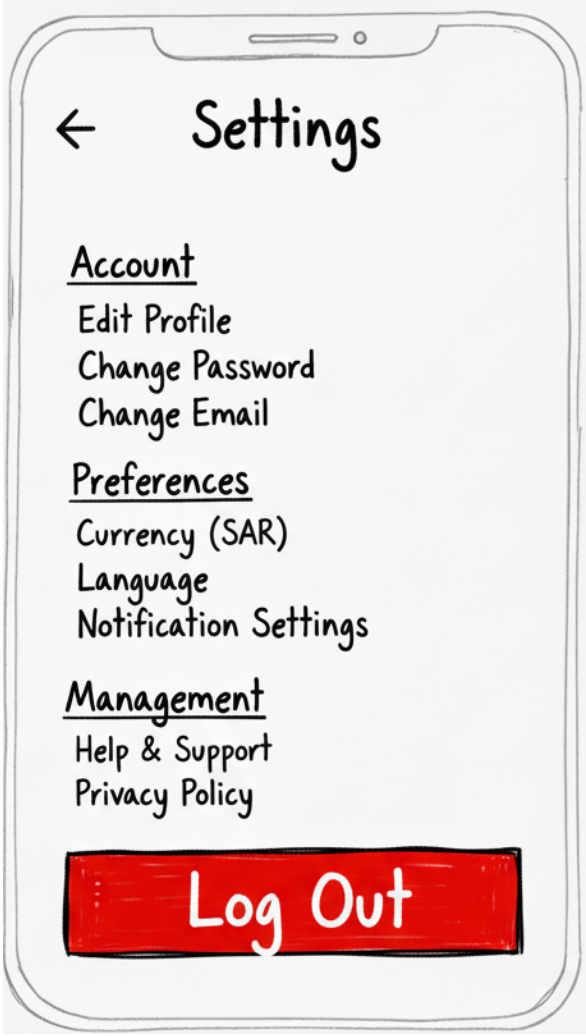


Screen 8

- **Budget Setup Screen**: Where the user sets their monthly spending limits.

Screen 9

- **Settings Screen**: Allows the user to manage their app preferences and account.



Application Flow

1. User opens the Mizan website.
2. The Login Screen is displayed.
3. A new user clicks "Sign Up", fills out the Registration Screen, and is returned to login.
4. User enters credentials and clicks "Login".
5. Upon successful authentication, the user is taken to the Dashboard.
6. From the Dashboard, the user can:
    a. Click "Add Expense/Income" to go to the form.
    b. Click the "Transactions" menu item to view the Transactions List.
    c. Click the "Reports" menu item to view charts.
    d. Click the "Budgets" menu item to set spending limits.
    e. Click "Settings" to manage their profile or categories.
7. The user can log out from any screen, ending their session and returning to the Login Screen.

Teamwork Strategy

Our team will use the following strategy to ensure smooth collaboration:

- **Roles**: We will assign loose roles based on strengths (e.g., Database Design, Front-end Ideas, Back-end Logic, Documentation) but everyone will help with all parts.
- **Meetings**: We will have two fixed meetings per week to discuss progress, solve problems, and plan next steps. Attendance is mandatory.
- **Communication**: We will use a WhatsApp group for quick, daily updates and questions. We will use Discord for longer discussions and sharing screens.
- **Task Management**: We will use a Trello board. We will create lists for "To Do", "In Progress", "Under Review", and "Done". Each task will be a card assigned to one or two people with a deadline.
- **Version Control**: We will use GitHub from the beginning to store and share all our code and SQL files. This allows us to work together without overwriting each other's work.
- **Documentation**: All documents (like this one) will be stored in a shared Google Drive folder, so everyone has the latest version.

Team Members' Contributions

All team members contributed with the discussion of all sections of this phase. But documenting them was give given as tasks to each of us.

Task	Member Responsible
Writing the Introduction	Norah Alfayez
Writing the Purpose and Scope	Reema Alkheraif
Writing the Data Requirements	Mashael Aljarbou
Writing the Functional Requirements	Jana Altalhi
Writing the Non-Functional Requirements	Norah Alfayez
Designing & Drawing the Paper Prototype	Reema Alkheraif, Jana Altalhi
Writing the Application Flow	Mashael Aljarbou
Writing the Teamwork Strategy	All Members
Reviewing everyone's work	All Members
Final Editing, Formatting, and Compilation	All Members

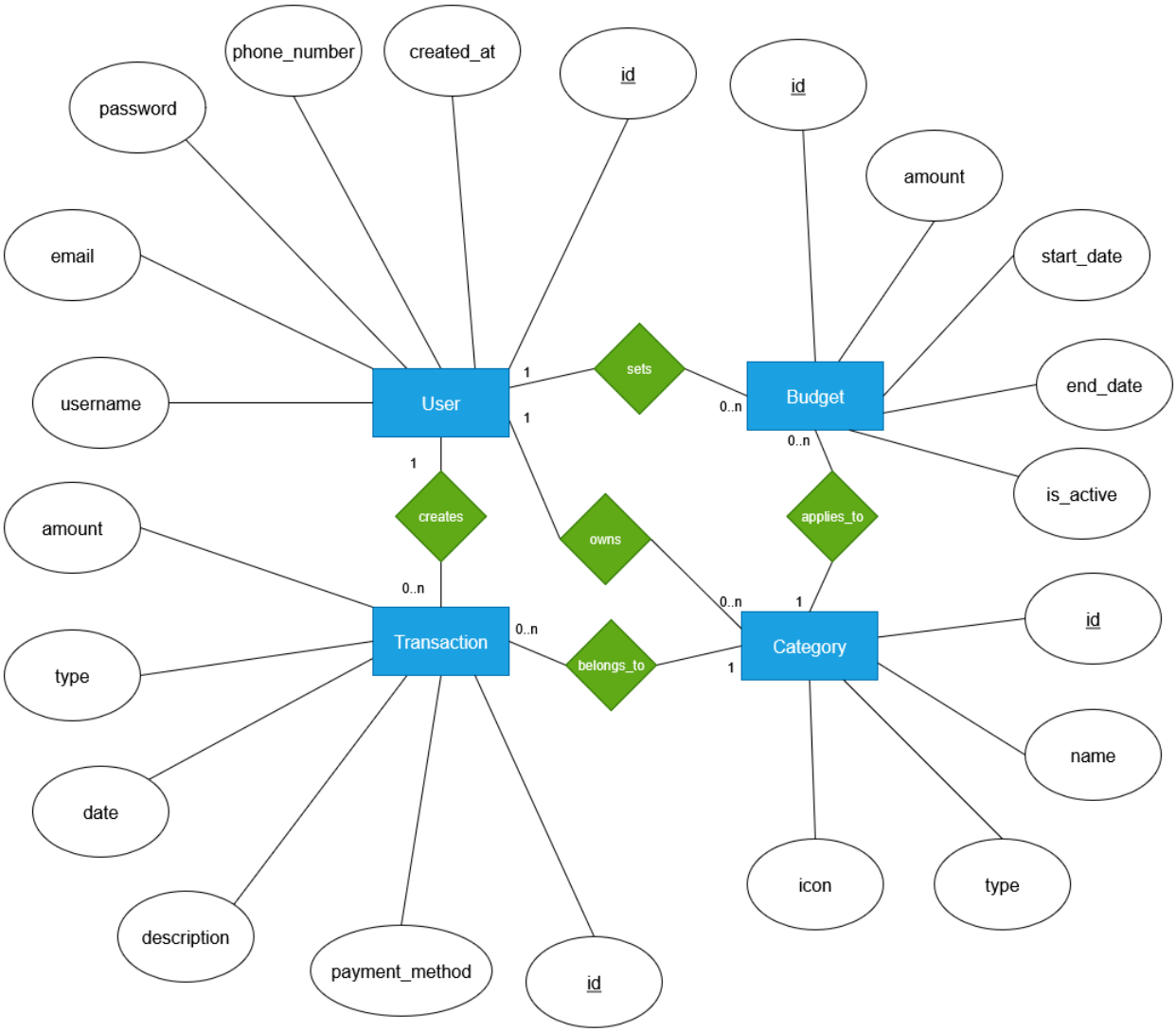
Mizan

Phase 2

CS 340 Introduction To Database Systems

Norah Alfayez - 222410215
Reema alkheraif - 222410580
Mashael aljarbou - 221410453
Jana Altalhi - 221410589

Enhanced Entity Relationship Diagram



Entity Sets and Attributes

Entity Set	Primary Key (PK)	Attributes	Description / Constraint Rationale
USER	user_id	username, email, password, phone_number (Optional), created_at	- The central entity. - Email is noted as a Unique/Candidate Key for authentication.
TRANSACTION	transaction_id	amount, type (Income/Expense), date,	- The fundamental financial record.

		description, payment_method	
CATEGORY	category_id	name, type (Income/Expense), icon	- Used to group transactions. - Name is unique per user.
BUDGET	budget_id	amount, start_date, end_date, is_active	- Defines spending limits over a time period.

Relationship Schema and Cardinality

User Ownership Relationships

The **USER** entity owns all other data structures.

Relationship Name	Participating Entities	Foreign Key (FK) Source	Cardinality	Constraint Rationale
Has	USER ↕ TRANSACTION	user_id in TRANSACTION	1:N (One User has Many Transactions)	A Transaction cannot exist without an owning User.
Owns	USER ↕ CATEGORY	user_id in CATEGORY	1:N (One User owns Many Categories)	Used to tell what user owns a category.
Sets	USER ↕ BUDGET	user_id in BUDGET	1:N (One User sets Many Budgets)	Every Budget must be linked to the User who created it.

Transaction/Budget Relationships

Relationship Name	Participating Entities	Foreign Key (FK) Source	Cardinality	Constraint Rationale
Belongs to	CATEGORY ↕ TRANSACTION	category_id in TRANSACTION	1:N (One Category has	All Transactions must be assigned a Category.

			Many Transactions)	
Applies to	CATEGORY ↕ BUDGET	category_id in BUDGET	1:N (One Category is targeted by Many Budgets)	This is a Partial Participation from the Budget side, as a Budget can be general (category_id can beNULL).

Team Members' Contributions

Task	Member
Design of the User and Core Entities	Norah Alfayez
Design of Transaction Entity and Linking	Reema Alkheraif
Design of Budget Entity and Constraints	Mashael Aljarbou
Design of Category Entity and Final EER Review	Jana Altalhi
Validation of EER Model against Requirements	All Members

Mizan

Phase 3

CS 340 Introduction To Database Systems

Dr.

Section:

Norah Alfayez - 222410215

Reema alkheraif - 222410580

Mashael aljarbou - 221410453

Jana Altalhi - 221410589

Data Dictionary

Below is the data dictionary for the Mizan database. It defines each table, its columns, data types, and constraints.

USER

Column Name	Data Type	Description	Constraints
user_id	INT	Unique ID for each user	Primary Key, Auto Increment
username	VARCHAR(100)	The username chosen by the user	Not Null
email	VARCHAR(255)	User's email address	Unique, Not Null
password	VARCHAR(255)	Encrypted user password	Not Null
phone_number	VARCHAR(20)	Optional phone number	Null
created_at	DATE	Date the user registered	Not Null

CATEGORY

Column Name	Data Type	Description	Constraints
category_id	INT	Unique ID for each category	Primary Key, Auto Increment
user_id	INT	Links to the user who owns this category	Foreign Key (user_id) references USER(user_id)
name	VARCHAR(100)	Category name (e.g., Food, Salary)	Not Null
type	VARCHAR(20)	Indicates if category is Income or Expense	Not Null
icon	VARCHAR(100)	Icon name for display (optional)	Null

TRANSACTION

Column Name	Data Type	Description	Constraints
transaction_id	INT	Unique ID for each transaction	Primary Key, Auto Increment
user_id	INT	Links to the user who created the transaction	Foreign Key (user_id) references USER(user_id)
category_id	INT	Links to the related category	Foreign Key (category_id) references CATEGORY(category_id)
amount	DECIMAL(10,2)	Amount of the transaction	Not Null

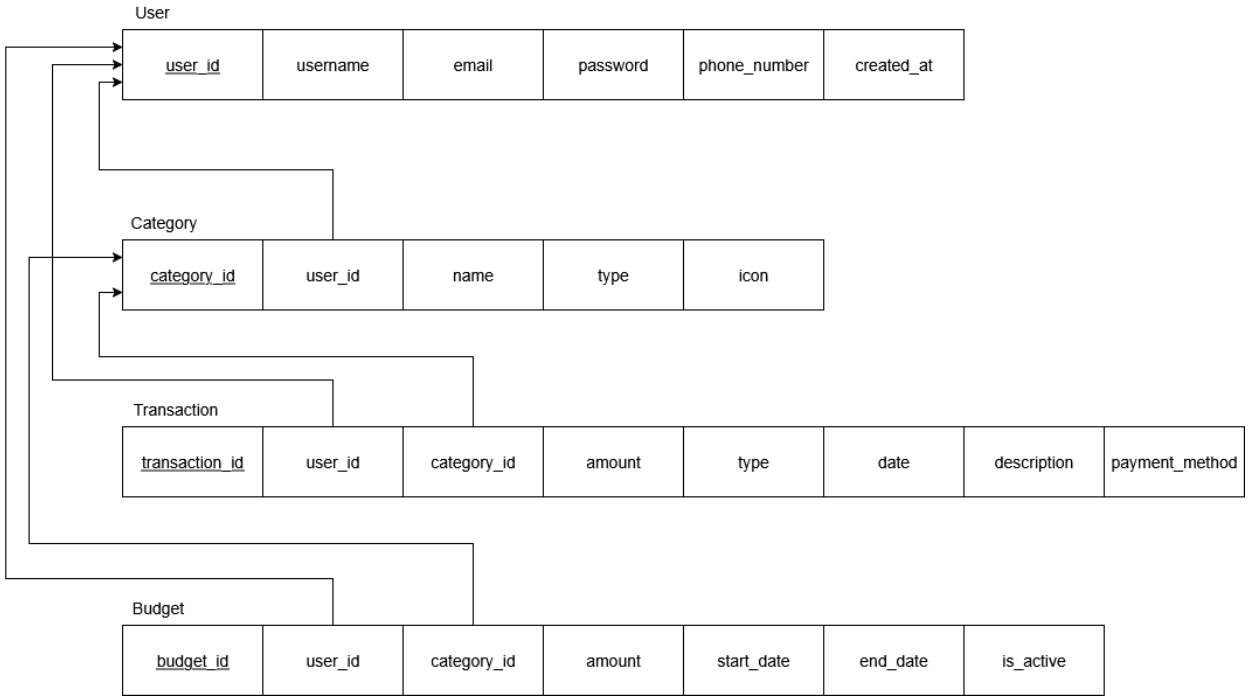
type	VARCHAR(20)	Income or Expense	Not Null
date	DATE	Date when the transaction occurred	Not Null
description	VARCHAR(255)	A short note for the transaction	Null
payment_method	VARCHAR(50)	Method of payment (Cash, Card, etc.)	Null

BUDGET

Column Name	Data Type	Description	Constraints
budget_id	INT	Unique ID for each budget	Primary Key, Auto Increment
user_id	INT	User who created this budget	Foreign Key (user_id) references USER(user_id)
category_id	INT	Category this budget applies to (optional)	Foreign Key (category_id) references CATEGORY(category_id), Null allowed
amount	DECIMAL(10,2)	Maximum spending amount	Not Null
start_date	DATE	Start date of the budget period	Not Null
end_date	DATE	End date of the budget period	Not Null
is_active	BOOLEAN	Indicates if the budget is currently active	Default TRUE

Relational Model

Below is the relational model showing how the tables connect:



Team Members' Contributions

Task	Member
Worked on defining the User and Category tables and their relationships.	Norah Alfayez
Helped design the Transaction table and ensure foreign key consistency.	Reema Alkheraif
Designed the Budget table and validated constraints between entities.	Mashael Aljarbou
Finalized the Relational Model Diagram and reviewed data dictionary formatting.	Jana Altalhi
Discussed and agreed on table structures, constraints, and keys during group meetings.	All Members

Mizan

Phase 4

CS 340 Introduction To Database Systems

Dr.

Section:

Norah Alfayez - 222410215

Reema alkheraif - 222410580

Mashael aljarbou - 221410453

Jana Altalhi - 221410589

SQL Table Creation

```
-- USER Table
CREATE TABLE USER (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20),
    created_at DATE NOT NULL
);

-- CATEGORY Table
CREATE TABLE CATEGORY (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('Income', 'Expense')),
    icon VARCHAR(100),
    FOREIGN KEY (user_id) REFERENCES USER(user_id) ON DELETE CASCADE
);

-- TRANSACTIONS Table
CREATE TABLE TRANSACTIONS (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    category_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    type VARCHAR(20) NOT NULL CHECK (type IN ('Income', 'Expense')),
    date DATE NOT NULL,
    description VARCHAR(255),
    payment_method VARCHAR(50),
    FOREIGN KEY (user_id) REFERENCES USER(user_id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id) ON DELETE CASCADE
);

-- BUDGET Table
CREATE TABLE BUDGET (
    budget_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    category_id INT NULL,
    amount DECIMAL(10,2) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (user_id) REFERENCES USER(user_id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id) ON DELETE SET NULL
);
```

After running the create queries, we can see that the tables were created.

Available Tables

BUDGET

budget_id	user_id	category_id	amount	start_date	end_date	is_active
empty						

CATEGORY

category_id	user_id	name	type	icon
empty				

TRANSACTIONS

transaction_id	user_id	category_id	amount	type	date	description	payment_meth
empty							

USER

user_id	username	email	password	phone_number	created_at
empty					

SQL Data Insertion

```
-- Insert Users (at least 10)
INSERT INTO USER (user_id, username, email, password, phone_number, created_at) VALUES
(1, 'ahmed_user', 'ahmed@email.com', 'hashed_password_1', '0501111111', '2025-01-15'),
(2, 'sara_finance', 'sara@email.com', 'hashed_password_2', '0502222222', '2025-01-16'),
(3, 'mohammed_sa', 'mohammed@email.com', 'hashed_password_3', '0503333333', '2025-01-17'),
(4, 'fatima_m', 'fatima@email.com', 'hashed_password_4', '0504444444', '2025-01-18'),
(5, 'khalid_2025', 'khalid@email.com', 'hashed_password_5', '0505555555', '2025-01-19'),
(6, 'nora_budget', 'nora@email.com', 'hashed_password_6', '0506666666', '2025-01-20'),
(7, 'faisal_money', 'faisal@email.com', 'hashed_password_7', '0507777777', '2025-01-21'),
(8, 'lama_saver', 'lama@email.com', 'hashed_password_8', '0508888888', '2025-01-22'),
(9, 'yousef_fin', 'yousef@email.com', 'hashed_password_9', '0509999999', '2025-01-23'),
(10, 'huda_personal', 'huda@email.com', 'hashed_password_10', '0501010101', '2025-01-24');

-- Insert Categories (at least 10 per user)
INSERT INTO CATEGORY (category_id, user_id, name, type, icon) VALUES
-- User 1 Categories
(1, 1, 'Salary', 'Income', 'salary-icon.png'),
(2, 1, 'Freelance', 'Income', 'freelance-icon.png'),
(3, 1, 'Food', 'Expense', 'food-icon.png'),
(4, 1, 'Transportation', 'Expense', 'transport-icon.png'),
(5, 1, 'Entertainment', 'Expense', 'entertainment-icon.png'),
(6, 1, 'Shopping', 'Expense', 'shopping-icon.png'),
(7, 1, 'Rent', 'Expense', 'rent-icon.png'),
(8, 1, 'Utilities', 'Expense', 'utilities-icon.png'),
```

```
(9, 1, 'Healthcare', 'Expense', 'health-icon.png'),
(10, 1, 'Education', 'Expense', 'education-icon.png'),

-- User 2 Categories
(11, 2, 'Salary', 'Income', 'salary-icon.png'),
(12, 2, 'Investment', 'Income', 'investment-icon.png'),
(13, 2, 'Food', 'Expense', 'food-icon.png'),
(14, 2, 'Transportation', 'Expense', 'transport-icon.png'),
(15, 2, 'Entertainment', 'Expense', 'entertainment-icon.png'),
(16, 2, 'Shopping', 'Expense', 'shopping-icon.png'),
(17, 2, 'Rent', 'Expense', 'rent-icon.png'),
(18, 2, 'Utilities', 'Expense', 'utilities-icon.png'),
(19, 2, 'Travel', 'Expense', 'travel-icon.png'),
(20, 2, 'Gifts', 'Expense', 'gifts-icon.png');

-- Insert Transactions (at least 10 per user)
INSERT INTO TRANSACTIONS (transaction_id, user_id, category_id, amount, type, date, description,
payment_method) VALUES
-- User 1 Transactions
(1, 1, 1, 8500.00, 'Income', '2025-03-01', 'Monthly Salary', 'Bank Transfer'),
(2, 1, 3, 45.50, 'Expense', '2025-03-02', 'Lunch at restaurant', 'Cash'),
(3, 1, 4, 30.00, 'Expense', '2025-03-03', 'Taxi fare', 'Cash'),
(4, 1, 5, 120.00, 'Expense', '2025-03-04', 'Movie tickets', 'Credit Card'),
(5, 1, 6, 250.75, 'Expense', '2025-03-05', 'Clothes shopping', 'Credit Card'),
(6, 1, 7, 2000.00, 'Expense', '2025-03-06', 'Monthly rent', 'Bank Transfer'),
(7, 1, 8, 350.00, 'Expense', '2025-03-07', 'Electricity bill', 'Bank Transfer'),
(8, 1, 9, 150.00, 'Expense', '2025-03-08', 'Doctor visit', 'Cash'),
(9, 1, 10, 300.00, 'Expense', '2025-03-09', 'Online course', 'Credit Card'),
(10, 1, 2, 500.00, 'Income', '2025-03-10', 'Freelance project', 'Bank Transfer'),

-- User 2 Transactions
(11, 2, 11, 12000.00, 'Income', '2025-03-01', 'Monthly Salary', 'Bank Transfer'),
(12, 2, 13, 35.00, 'Expense', '2025-03-02', 'Groceries', 'Cash'),
(13, 2, 14, 25.00, 'Expense', '2025-03-03', 'Bus fare', 'Cash'),
(14, 2, 15, 80.00, 'Expense', '2025-03-04', 'Concert tickets', 'Credit Card'),
(15, 2, 16, 180.50, 'Expense', '2025-03-05', 'Shoes', 'Credit Card'),
(16, 2, 17, 2500.00, 'Expense', '2025-03-06', 'Apartment rent', 'Bank Transfer'),
(17, 2, 18, 280.00, 'Expense', '2025-03-07', 'Water and internet', 'Bank Transfer'),
(18, 2, 19, 500.00, 'Expense', '2025-03-08', 'Weekend trip', 'Credit Card'),
(19, 2, 20, 75.00, 'Expense', '2025-03-09', 'Birthday gift', 'Cash'),
(20, 2, 12, 300.00, 'Income', '2025-03-10', 'Stock dividends', 'Bank Transfer');

-- Insert Budgets (at least 10)
INSERT INTO BUDGET (budget_id, user_id, category_id, amount, start_date, end_date, is_active) VALUES
-- User 1 Budgets
(1, 1, NULL, 5000.00, '2025-03-01', '2025-03-31', TRUE),
(2, 1, 3, 800.00, '2025-03-01', '2025-03-31', TRUE),
(3, 1, 4, 300.00, '2025-03-01', '2025-03-31', TRUE),
(4, 1, 5, 500.00, '2025-03-01', '2025-03-31', TRUE),
(5, 1, 6, 1000.00, '2025-03-01', '2025-03-31', TRUE),

-- User 2 Budgets
(6, 2, NULL, 7000.00, '2025-03-01', '2025-03-31', TRUE),
(7, 2, 13, 600.00, '2025-03-01', '2025-03-31', TRUE),
(8, 2, 14, 200.00, '2025-03-01', '2025-03-31', TRUE),
(9, 2, 15, 400.00, '2025-03-01', '2025-03-31', TRUE),
```

```
(10, 2, 16, 800.00, '2025-03-01', '2025-03-31', TRUE);
```

After running the queries, we can see the data in the tables.

USER

user_id	username	email	password	phone_number	created_at
1	ahmed_user	ahmed@email.com	hashed_password_1	0501111111	2025-01-15
2	sara_finance	sara@email.com	hashed_password_2	0502222222	2025-01-16
3	mohammed_sa	mohammed@email.com	hashed_password_3	0503333333	2025-01-17
4	fatima_m	fatima@email.com	hashed_password_4	0504444444	2025-01-18
5	khalid_2025	khalid@email.com	hashed_password_5	0505555555	2025-01-19
6	nora_budget	nora@email.com	hashed_password_6	0506666666	2025-01-20
7	faisal_money	faisal@email.com	hashed_password_7	0507777777	2025-01-21
8	lama_saver	lama@email.com	hashed_password_8	0508888888	2025-01-22
9	yousef_fin	yousef@email.com	hashed_password_9	0509999999	2025-01-23
10	huda_personal	huda@email.com	hashed_password_10	0501010101	2025-01-24

CATEGORY

category_id	user_id	name	type	icon
1	1	Salary	Income	salary-icon.png
2	1	Freelance	Income	freelance-icon.png
3	1	Food	Expense	food-icon.png
4	1	Transportation	Expense	transport-icon.png
5	1	Entertainment	Expense	entertainment-icon.png
6	1	Shopping	Expense	shopping-icon.png
7	1	Rent	Expense	rent-icon.png
8	1	Utilities	Expense	utilities-icon.png
9	1	Healthcare	Expense	health-icon.png
10	1	Education	Expense	education-icon.png
11	2	Salary	Income	salary-icon.png
12	2	Investment	Income	investment-icon.png
13	2	Food	Expense	food-icon.png
14	2	Transportation	Expense	transport-icon.png
15	2	Entertainment	Expense	entertainment-icon.png
16	2	Shopping	Expense	shopping-icon.png
17	2	Rent	Expense	rent-icon.png
18	2	Utilities	Expense	utilities-icon.png
19	2	Travel	Expense	travel-icon.png
20	2	Gifts	Expense	gifts-icon.png

TRANSACTIONS

transaction_id	user_id	category_id	amount	type	date	description	payment_method
1	1	1	8500	Income	2025-03-01	Monthly Salary	Bank Transfer
2	1	3	45.5	Expense	2025-03-02	Lunch at restaurant	Cash
3	1	4	30	Expense	2025-03-03	Taxi fare	Cash
4	1	5	120	Expense	2025-03-04	Movie tickets	Credit Card
5	1	6	250.75	Expense	2025-03-05	Clothes shopping	Credit Card
6	1	7	2000	Expense	2025-03-06	Monthly rent	Bank Transfer
7	1	8	350	Expense	2025-03-07	Electricity bill	Bank Transfer
8	1	9	150	Expense	2025-03-08	Doctor visit	Cash
9	1	10	300	Expense	2025-03-09	Online course	Credit Card
10	1	2	500	Income	2025-03-10	Freelance project	Bank Transfer
11	2	11	12000	Income	2025-03-01	Monthly Salary	Bank Transfer
12	2	13	35	Expense	2025-03-02	Groceries	Cash
13	2	14	25	Expense	2025-03-03	Bus fare	Cash
14	2	15	80	Expense	2025-03-04	Concert tickets	Credit Card
15	2	16	180.5	Expense	2025-03-05	Shoes	Credit Card
16	2	17	2500	Expense	2025-03-06	Apartment rent	Bank Transfer
17	2	18	280	Expense	2025-03-07	Water and internet	Bank Transfer
18	2	19	500	Expense	2025-03-08	Weekend trip	Credit Card
19	2	20	75	Expense	2025-03-09	Birthday gift	Cash
20	2	12	300	Income	2025-03-10	Stock dividends	Bank Transfer

BUDGET

budget_id	user_id	category_id	amount	start_date	end_date	is_active
1	1		5000	2025-03-01	2025-03-31	1
2	1	3	800	2025-03-01	2025-03-31	1
3	1	4	300	2025-03-01	2025-03-31	1
4	1	5	500	2025-03-01	2025-03-31	1
5	1	6	1000	2025-03-01	2025-03-31	1
6	2		7000	2025-03-01	2025-03-31	1
7	2	13	600	2025-03-01	2025-03-31	1
8	2	14	200	2025-03-01	2025-03-31	1
9	2	15	400	2025-03-01	2025-03-31	1
10	2	16	800	2025-03-01	2025-03-31	1

Team Members' Contributions

Task	Member
SQL Table Creation Script	Norah Alfayez
User & Category Sample Data	Reema Alkheraif
Transaction Sample Data	Mashael Aljarbou
Budget Sample Data & Constraints	Jana Altalhi

-	**SQL Syntax Validation & Testing**	All Members
-	**Documentation & Final Review**	

Mizan

Phase 5

CS 340 Introduction To Database Systems

Dr.

Section:

Norah Alfayez - 222410215

Reema alkheraif - 222410580

Mashael aljarbou - 221410453

Jana Altalhi - 221410589

Database SQL Queries

Norah Alfayez

Basic Queries

Get all users with their account creation date

```
SELECT user_id, username, email, created_at
FROM USER
ORDER BY created_at DESC;
```

user_id	username	email	created_at
10	huda_personal	huda@email.com	2025-01-24
9	yousef_fin	yousef@email.com	2025-01-23
8	lama_saver	lama@email.com	2025-01-22
7	faisal_money	faisal@email.com	2025-01-21
6	nora_budget	nora@email.com	2025-01-20
5	khalid_2025	khalid@email.com	2025-01-19
4	fatima_m	fatima@email.com	2025-01-18
3	mohammed_sa	mohammed@email.com	2025-01-17
2	sara_finance	sara@email.com	2025-01-16
1	ahmed_user	ahmed@email.com	2025-01-15

Count total transactions per user

```
SELECT u.username, COUNT(t.transaction_id) as total_transactions
FROM USER u
LEFT JOIN TRANSACTIONS t ON u.user_id = t.user_id
GROUP BY u.user_id, u.username;
```

username	total_transactions
ahmed_user	10
sara_finance	10
mohammed_sa	0
fatima_m	0
khalid_2025	0
nora_budget	0
faisal_money	0
lama_saver	0
yousef_fin	0
huda_personal	0

Find all expense categories for a specific user

```
SELECT name, icon
FROM CATEGORY
WHERE user_id = 1 AND type = 'Expense';
```

name	icon
Food	food-icon.png
Transportation	transport-icon.png
Entertainment	entertainment-icon.png
Shopping	shopping-icon.png
Rent	rent-icon.png
Utilities	utilities-icon.png
Healthcare	health-icon.png
Education	education-icon.png

Get recent transactions with category names

```
SELECT t.amount, t.date, t.description, c.name as category_name
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
WHERE t.user_id = 1
ORDER BY t.date DESC
LIMIT 10;
```

amount	date	description	category_name
500	2025-03-10	Freelance project	Freelance
300	2025-03-09	Online course	Education
150	2025-03-08	Doctor visit	Healthcare
350	2025-03-07	Electricity bill	Utilities
2000	2025-03-06	Monthly rent	Rent
250.75	2025-03-05	Clothes shopping	Shopping
120	2025-03-04	Movie tickets	Entertainment
30	2025-03-03	Taxi fare	Transportation
45.5	2025-03-02	Lunch at restaurant	Food
8500	2025-03-01	Monthly Salary	Salary

Find active budgets for a user

```
SELECT b.amount, b.start_date, b.end_date, c.name as category_name
FROM BUDGET b
LEFT JOIN CATEGORY c ON b.category_id = c.category_id
WHERE b.user_id = 1 AND b.is_active = TRUE;
```

amount	start_date	end_date	category_name
5000	2025-03-01	2025-03-31	
800	2025-03-01	2025-03-31	Food
300	2025-03-01	2025-03-31	Transportation
500	2025-03-01	2025-03-31	Entertainment
1000	2025-03-01	2025-03-31	Shopping

Advanced Queries

Monthly income vs expense summary

```
SELECT
    strftime('%Y', date) as year,
    strftime('%m', date) as month,
    SUM(CASE WHEN type = 'Income' THEN amount ELSE 0 END) as total_income,
    SUM(CASE WHEN type = 'Expense' THEN amount ELSE 0 END) as total_expense,
    SUM(CASE WHEN type = 'Income' THEN amount ELSE -amount END) as net_savings
FROM TRANSACTIONS
WHERE user_id = 1
GROUP BY strftime('%Y', date), strftime('%m', date)
ORDER BY year DESC, month DESC;
```

year	month	total_income	total_expense	net_savings
2025	03	9000	3246.25	5753.75

Find categories where spending exceeds budget

```
SELECT
c.category_id,
    c.name as category_name,
    SUM(t.amount) as actual_spent,
    b.amount as budget_amount,
    (b.amount - SUM(t.amount)) as remaining_budget
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
JOIN BUDGET b ON c.category_id = b.category_id
WHERE t.user_id = 1
  AND t.type = 'Expense'
  AND t.date BETWEEN b.start_date AND b.end_date
  AND b.is_active = 1
GROUP BY c.category_id, c.name, b.amount
HAVING SUM(t.amount) > b.amount;
```

category_id	category_name	actual_spent	budget_amount	remaining_budget
10	Education	100000	7000	-93000

Most used payment methods

```
SELECT
    payment_method,
    COUNT(*) as transaction_count,
    SUM(amount) as total_amount
FROM TRANSACTIONS
WHERE user_id = 1
GROUP BY payment_method
ORDER BY transaction_count DESC;
```

payment_method	transaction_count	total_amount
Bank Transfer	4	11350
Credit Card	3	100370.75
Cash	3	225.5

Weekly spending pattern

```
SELECT
    strftime('%W', date) as week,
    SUM(amount) as weekly_total,
    COUNT(*) as transaction_count
FROM TRANSACTIONS
WHERE user_id = 1 AND type = 'Expense'
GROUP BY strftime('%Y-%W', date)
ORDER BY week DESC
LIMIT 8;
```

week	weekly_total	transaction_count
09	102900.75	7
08	45.5	1

Categories with highest average transaction value

```
SELECT
    c.name as category_name,
    AVG(t.amount) as avg_transaction,
    MAX(t.amount) as highest_transaction,
    COUNT(*) as transaction_count
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
WHERE t.user_id = 1 AND t.type = 'Expense'
GROUP BY c.category_id, c.name
ORDER BY avg_transaction DESC;
```

category_name	avg_transaction	highest_transaction	transaction_count
Education	100000	100000	1
Rent	2000	2000	1
Utilities	350	350	1
Shopping	250.75	250.75	1
Healthcare	150	150	1
Entertainment	120	120	1
Food	45.5	45.5	1
Transportation	30	30	1

Reema Alkheraif

Basic Queries

List all budgets with category names

```
SELECT b.budget_id, c.name, b.amount, b.start_date, b.end_date
FROM BUDGET b
LEFT JOIN CATEGORY c ON b.category_id = c.category_id
WHERE b.user_id = 1;
```

budget_id	name	amount	start_date	end_date
1		5000	2025-03-01	2025-03-31
2	Food	800	2025-03-01	2025-03-31
3	Transportation	300	2025-03-01	2025-03-31
4	Entertainment	500	2025-03-01	2025-03-31
5	Shopping	1000	2025-03-01	2025-11-11

Find active budgets

```
SELECT budget_id, amount, start_date, end_date
FROM BUDGET
WHERE user_id = 1 AND is_active = 1;
```

budget_id	amount	start_date	end_date
1	5000	2025-03-01	2025-03-31
2	800	2025-03-01	2025-03-31
3	300	2025-03-01	2025-03-31
4	500	2025-03-01	2025-03-31
5	1000	2025-03-01	2025-11-11

Count budgets per user

```
SELECT user_id, COUNT(*) as budget_count
FROM BUDGET
GROUP BY user_id;
```

user_id	budget_count
1	5
2	5

Find general budgets

```
SELECT budget_id, amount, start_date, end_date
FROM BUDGET
WHERE user_id = 1 AND category_id IS NULL;
```

budget_id	amount	start_date	end_date
1	5000	2025-03-01	2025-03-31

Get total budget amount per user

```
SELECT user_id, SUM(amount) as total_budget
FROM BUDGET
WHERE is_active = 1
GROUP BY user_id;
```

user_id	total_budget
1	7600
2	9000

Advanced Queries

Budget vs actual spending

```
SELECT
    b.budget_id,
    c.name as category_name,
    b.amount as budget_amount,
    (SELECT SUM(amount) FROM TRANSACTIONS t
     WHERE t.category_id = b.category_id
     AND t.date >= b.start_date AND t.date <= b.end_date) as actual_spent
FROM BUDGET b
LEFT JOIN CATEGORY c ON b.category_id = c.category_id
WHERE b.user_id = 1 AND b.is_active = 1;
```

budget_id	category_name	budget_amount	actual_spent
1		5000	
2	Food	800	45.5
3	Transportation	300	30
4	Entertainment	500	100120
5	Shopping	1000	250.75

Over budget categories

```
SELECT
    c.name as category_name,
    b.amount as budget_limit,
    (SELECT SUM(amount) FROM TRANSACTIONS t
     WHERE t.category_id = c.category_id
     AND t.date >= b.start_date AND t.date <= b.end_date) as spent
FROM BUDGET b
JOIN CATEGORY c ON b.category_id = c.category_id
WHERE b.user_id = 1
AND (SELECT SUM(amount) FROM TRANSACTIONS t
     WHERE t.category_id = c.category_id
     AND t.date >= b.start_date AND t.date <= b.end_date) > b.amount;
```

category_name	budget_limit	spent
Entertainment	500	100120

Budgets ending soon

```
SELECT budget_id, amount, end_date
FROM BUDGET
WHERE user_id = 1 AND is_active = 1
```

```
AND end_date > date('now')
ORDER BY end_date ASC
LIMIT 5;
```

budget_id	amount	end_date
5	1000	2025-11-11

User's total budget amount

```
SELECT user_id, SUM(amount) as total_budget
FROM BUDGET
WHERE is_active = 1
GROUP BY user_id;
```

user_id	total_budget
1	7600
2	9000

Budgets by category type

```
SELECT
   c.type,
   COUNT(b.budget_id) as budget_count
FROM BUDGET b
JOIN CATEGORY c ON b.category_id = c.category_id
WHERE b.user_id = 1
GROUP BY c.type;
```

type	budget_count
Expense	4

Mashael aljarbou

Basic Queries

Get recent transactions

```
SELECT t.amount, t.type, t.date, t.description, c.name
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
WHERE t.user_id = 1
ORDER BY t.date DESC
LIMIT 10;
```

amount	type	date	description	name
500	Income	2025-03-10	Freelance project	Freelance
100000	Expense	2025-03-09	Online course	Education
150	Expense	2025-03-08	Doctor visit	Healthcare
350	Expense	2025-03-07	Electricity bill	Utilities
2000	Expense	2025-03-06	Monthly rent	Rent
250.75	Expense	2025-03-05	Clothes shopping	Shopping
120	Expense	2025-03-04	Movie tickets	Entertainment
30	Expense	2025-03-03	Taxi fare	Transportation
45.5	Expense	2025-03-02	Lunch at restaurant	Food
8500	Income	2025-03-01	Monthly Salary	Salary

Find largest expenses

```
SELECT amount, date, description
FROM TRANSACTIONS
WHERE user_id = 1 AND type = 'Expense'
ORDER BY amount DESC
LIMIT 5;
```

amount	date	description
100000	2025-03-09	Online course
2000	2025-03-06	Monthly rent
350	2025-03-07	Electricity bill
250.75	2025-03-05	Clothes shopping
150	2025-03-08	Doctor visit

Count transactions by type

```
SELECT type, COUNT(*) as count
FROM TRANSACTIONS
WHERE user_id = 1
GROUP BY type;
```

type	count
Expense	8
Income	2

Find transactions with descriptions

```
SELECT transaction_id, amount, date, description
FROM TRANSACTIONS
WHERE user_id = 1 AND description IS NOT NULL;
```

transaction_id	amount	date	description
1	8500	2025-03-01	Monthly Salary
2	45.5	2025-03-02	Lunch at restaurant
3	30	2025-03-03	Taxi fare
4	120	2025-03-04	Movie tickets
5	250.75	2025-03-05	Clothes shopping
6	2000	2025-03-06	Monthly rent
7	350	2025-03-07	Electricity bill
8	150	2025-03-08	Doctor visit
9	100000	2025-03-09	Online course
10	500	2025-03-10	Freelance project

Get total transactions per day

```
SELECT date, COUNT(*) as count
FROM TRANSACTIONS
WHERE user_id = 1
GROUP BY date;
```

date	count
2025-03-01	1
2025-03-02	1
2025-03-03	1
2025-03-04	1
2025-03-05	1
2025-03-06	1
2025-03-07	1
2025-03-08	1
2025-03-09	1
2025-03-10	1

Advanced Queries

Monthly totals

```
SELECT
   substr(date, 1, 7) as month,
   SUM(amount) as total
FROM TRANSACTIONS
WHERE user_id = 1 AND type = 'Expense'
GROUP BY substr(date, 1, 7)
ORDER BY month DESC;
```

month	total
2025-03	102946.25

Top spending categories

```
SELECT
   c.name,
```

```
    SUM(t.amount) as total_spent
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
WHERE t.user_id = 1 AND t.type = 'Expense'
GROUP BY c.name
ORDER BY total_spent DESC
LIMIT 5;
```

name	total_spent
Entertainment	100120
Rent	2000
Utilities	350
Shopping	250.75
Healthcare	150

Payment method summary

```
SELECT
    payment_method,
    COUNT(*) as count,
    SUM(amount) as total
FROM TRANSACTIONS
WHERE user_id = 1 AND payment_method IS NOT NULL
GROUP BY payment_method;
```

payment_method	count	total
Bank Transfer	4	11350
Cash	3	225.5
Credit Card	3	100370.75

Recent high expenses

```
SELECT
    amount,
    date,
    description,
    (SELECT name FROM CATEGORY c WHERE c.category_id = t.category_id) as category
FROM TRANSACTIONS t
WHERE user_id = 1 AND type = 'Expense' AND amount > 100
ORDER BY date DESC;
```

amount	date	description	category
100000	2025-03-09	Online course	Entertainment
150	2025-03-08	Doctor visit	Healthcare
350	2025-03-07	Electricity bill	Utilities
2000	2025-03-06	Monthly rent	Rent
250.75	2025-03-05	Clothes shopping	Shopping
120	2025-03-04	Movie tickets	Entertainment

Category averages

```
SELECT
    c.name,
    AVG(t.amount) as avg_amount,
    COUNT(*) as count
FROM TRANSACTIONS t
JOIN CATEGORY c ON t.category_id = c.category_id
WHERE t.user_id = 1
GROUP BY c.name;
```

name	avg_amount	count
Entertainment	50060	2
Food	45.5	1
Freelance	500	1
Healthcare	150	1
Rent	2000	1
Salary	8500	1
Shopping	250.75	1
Transportation	30	1
Utilities	350	1

Jana Altalhi

Basic Queries

Find users with their transaction counts

```
SELECT u.username, COUNT(t.transaction_id) as transaction_count
FROM USER u
LEFT JOIN TRANSACTIONS t ON u.user_id = t.user_id
GROUP BY u.user_id, u.username;
```

username	transaction_count
ahmed_user	10
sara_finance	10
mohammed_sa	0
fatima_m	0
khalid_2025	0
nora_budget	0
faisal_money	0
lama_saver	0
yousef_fin	0
huda_personal	0

List categories with usage count

```
SELECT c.name, c.type, COUNT(t.transaction_id) as usage_count
FROM CATEGORY c
LEFT JOIN TRANSACTIONS t ON c.category_id = t.category_id
WHERE c.user_id = 1
GROUP BY c.name, c.type;
```

name	type	usage_count
Education	Expense	1
Entertainment	Expense	1
Food	Expense	1
Freelance	Income	1
Healthcare	Expense	1
Rent	Expense	1
Salary	Income	1
Shopping	Expense	1
Transportation	Expense	1
Utilities	Expense	1

Find unused categories

```
SELECT c.name, c.type
FROM CATEGORY c
LEFT JOIN TRANSACTIONS t ON c.category_id = t.category_id
WHERE c.user_id = 1 AND t.transaction_id IS NULL;
```

name	type
Education	Expense

Get user registration dates

```
SELECT username, email, created_at
FROM USER
ORDER BY created_at DESC;
```

username	email	created_at
huda_personal	huda@email.com	2025-01-24
yousef_fin	yousef@email.com	2025-01-23
lama_saver	lama@email.com	2025-01-22
faisal_money	faisal@email.com	2025-01-21
nora_budget	nora@email.com	2025-01-20
khalid_2025	khalid@email.com	2025-01-19
fatima_m	fatima@email.com	2025-01-18
mohammed_sa	mohammed@email.com	2025-01-17
sara_finance	sara@email.com	2025-01-16
ahmed_user	ahmed@email.com	2025-01-15

List expense categories only

```
SELECT name, icon
FROM CATEGORY
WHERE user_id = 1 AND type = 'Expense';
```

name	icon
Food	food-icon.png
Transportation	transport-icon.png
Entertainment	entertainment-icon.png
Shopping	shopping-icon.png
Rent	rent-icon.png

Advanced Queries

Most active users

```
SELECT
    username,
    (SELECT COUNT(*) FROM TRANSACTIONS t WHERE t.user_id = u.user_id) as
transaction_count
FROM USER u
ORDER BY transaction_count DESC;
```

username	transaction_count
ahmed_user	10
sara_finance	10
mohammed_sa	0
fatima_m	0
khalid_2025	0
nora_budget	0
faisal_money	0
lama_saver	0
yousef_fin	0
huda_personal	0

Popular categories

```
SELECT
    c.name,
    c.type,
    (SELECT COUNT(*) FROM TRANSACTIONS t WHERE t.category_id = c.category_id)
as usage_count
FROM CATEGORY c
WHERE c.user_id = 1
ORDER BY usage_count DESC;
```

name	type	usage_count
Entertainment	Expense	2
Salary	Income	1
Freelance	Income	1
Food	Expense	1
Transportation	Expense	1
Shopping	Expense	1
Rent	Expense	1
Utilities	Expense	1
Healthcare	Expense	1
Education	Expense	0

User financial summary

```
SELECT
    u.username,
    (SELECT SUM(amount) FROM TRANSACTIONS t WHERE t.user_id = u.user_id AND
t.type = 'Income') as total_income,
    (SELECT SUM(amount) FROM TRANSACTIONS t WHERE t.user_id = u.user_id AND
t.type = 'Expense') as total_expense
FROM USER u;
```

username	total_income	total_expense
ahmed_user	9000	102946.25
sara_finance	12300	3675.5
mohammed_sa		
fatima_m		
khalid_2025		
nora_budget		
faisal_money		
lama_saver		
yousef_fin		
huda_personal		

Category creation order

```
SELECT
    name,
    type,
    (SELECT COUNT(*) FROM TRANSACTIONS t WHERE t.category_id = c.category_id)
as times_used
FROM CATEGORY c
WHERE user_id = 1
ORDER BY times_used DESC;
```

name	type	times_used
Entertainment	Expense	2
Salary	Income	1
Freelance	Income	1
Food	Expense	1
Transportation	Expense	1
Shopping	Expense	1
Rent	Expense	1
Utilities	Expense	1
Healthcare	Expense	1
Education	Expense	0

Users with most categories

```
SELECT
    u.username,
    (SELECT COUNT(*) FROM CATEGORY c WHERE c.user_id = u.user_id) as
category_count
FROM USER u
ORDER BY category_count DESC;
```

username	category_count
ahmed_user	10
sara_finance	10
mohammed_sa	0
fatima_m	0
khalid_2025	0
nora_budget	0
faisal_money	0
lama_saver	0
yousef_fin	0
huda_personal	0

Java Front and Back Ends

To allow the user to run the SQL Queries, we created a Java Swing application. The frontend gives the user multiple forms and panels where he can enter some data or preview it when opening any page. In the back, we connected each action done by the user to a collection of Java classes that handle the connections and the data validation.

To manage the security, we don't store the passwords as plain text. We store their hashes. And in the queries we don't concatenate the values directly to the queries to avoid SQL injection. We bind them as parameters.

On the backend, the application uses standard JDBC driver to connect to the database. To make the SQL and database management easier, we have created a class that handles the database configurations and all the SQL statements. Here is the Database Helper class:

```
package mizan;

import mizan.model.Category;
import mizan.model.TransactionRecord;
import mizan.model.User;

import java.math.BigDecimal;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.sql.Connection;
import java.sql.Date;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.time.YearMonth;
import java.util.ArrayList;
import java.util.Base64;
import java.util.List;
import java.util.Optional;

public class DatabaseHelper {

  private static final String DB_URL = "jdbc:mysql://localhost:3306/mizan";
  private static final String DB_USERNAME = "root";
  private static final String DB_PASSWORD = "";

  private static final DatabaseHelper INSTANCE = new DatabaseHelper();

  private DatabaseHelper() {
    try {
      Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
      throw new IllegalStateException("MySQL Driver not found.", e);
    }
  }

  public static DatabaseHelper getInstance() {
    return INSTANCE;
  }

  private Connection getConnection() throws SQLException {
    return DriverManager.getConnection(DB_URL, DB_USERNAME, DB_PASSWORD);
  }

  private String hashPassword(String password) {
    try {
```

```
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(password.getBytes());
        return Base64.getEncoder().encodeToString(hash);
      } catch (NoSuchAlgorithmException e) {
        throw new IllegalStateException("SHA-256 algorithm not available", e);
      }
    }

    public boolean registerUser(String username, String email, String password, String phoneNumber) {
      String sql = "INSERT INTO USER (username, email, password, phone_number, created_at) VALUES (?,
?, ?, ?, CURDATE())";
      try (Connection connection = getConnection();
           PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, username);
        statement.setString(2, email);
        statement.setString(3, hashPassword(password));
        statement.setString(4, phoneNumber);

        return statement.executeUpdate() == 1;
      } catch (SQLException ex) {
        ex.printStackTrace();
        return false;
      }
    }

    public Optional<User> login(String email, String password) {
      String sql = "SELECT * FROM USER WHERE email = ? AND password = ?";
      try (Connection connection = getConnection();
           PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setString(1, email);
        statement.setString(2, hashPassword(password));

        ResultSet rs = statement.executeQuery();
        if (rs.next()) {
          User user = new User(
              rs.getInt("user_id"),
              rs.getString("username"),
              rs.getString("email"),
              rs.getString("password"),
              rs.getString("phone_number"),
              rs.getDate("created_at").toLocalDate()
          );
          return Optional.of(user);
        }
        return Optional.empty();
      } catch (SQLException ex) {
        ex.printStackTrace();
        return Optional.empty();
      }
    }
```

```
  public List<Category> getCategories(int userId) {
    String sql = "SELECT category_id, user_id, name, type, icon FROM CATEGORY WHERE user_id = ?
ORDER BY name";
    List<Category> categories = new ArrayList<>();
    try (Connection connection = getConnection();
        PreparedStatement statement = connection.prepareStatement(sql)) {

      statement.setInt(1, userId);
      ResultSet rs = statement.executeQuery();
      while (rs.next()) {
        categories.add(new Category(
            rs.getInt("category_id"),
            rs.getInt("user_id"),
            rs.getString("name"),
            rs.getString("type"),
            rs.getString("icon")
        ));
      }
    } catch (SQLException ex) {
      ex.printStackTrace();
    }
    return categories;
  }

  public boolean addTransaction(TransactionRecord record) {
    String sql = "INSERT INTO TRANSACTIONS (user_id, category_id, amount, type, date, description,
payment_method) "
        + "VALUES (?, ?, ?, ?, ?, ?, ?)";
    try (Connection connection = getConnection();
        PreparedStatement statement = connection.prepareStatement(sql)) {

      statement.setInt(1, record.userId());
      statement.setInt(2, record.categoryId());
      statement.setBigDecimal(3, record.amount());
      statement.setString(4, record.type());
      statement.setDate(5, Date.valueOf(record.date()));
      statement.setString(6, record.description());
      statement.setString(7, record.paymentMethod());
      return statement.executeUpdate() == 1;
    } catch (SQLException ex) {
      ex.printStackTrace();
      return false;
    }
  }

  public List<TransactionRecord> getTransactions(int userId) {
    String sql = "SELECT transaction_id, user_id, category_id, amount, type, date, description,
payment_method "
        + "FROM TRANSACTIONS WHERE user_id = ? ORDER BY date DESC";
    List<TransactionRecord> transactions = new ArrayList<>();
    try (Connection connection = getConnection();
        PreparedStatement statement = connection.prepareStatement(sql)) {
```

```
        statement.setInt(1, userId);
        ResultSet rs = statement.executeQuery();
        while (rs.next()) {
          transactions.add(new TransactionRecord(
              rs.getInt("transaction_id"),
              rs.getInt("user_id"),
              rs.getInt("category_id"),
              rs.getBigDecimal("amount"),
              rs.getString("type"),
              rs.getDate("date").toLocalDate(),
              rs.getString("description"),
              rs.getString("payment_method")
          ));
        }
      } catch (SQLException ex) {
        ex.printStackTrace();
      }
      return transactions;
    }

    public boolean updateTransaction(TransactionRecord record) {
      String sql = "UPDATE TRANSACTIONS SET amount = ?, category_id = ?, type = ?, date = ?, description =
?, payment_method = ? "
          + "WHERE transaction_id = ?";
      try (Connection connection = getConnection();
          PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setBigDecimal(1, record.amount());
        statement.setInt(2, record.categoryId());
        statement.setString(3, record.type());
        statement.setDate(4, Date.valueOf(record.date()));
        statement.setString(5, record.description());
        statement.setString(6, record.paymentMethod());
        statement.setInt(7, record.transactionId());

        return statement.executeUpdate() == 1;
      } catch (SQLException ex) {
        ex.printStackTrace();
        return false;
      }
    }

    public boolean deleteTransaction(int transactionId) {
      String sql = "DELETE FROM TRANSACTIONS WHERE transaction_id = ?";
      try (Connection connection = getConnection();
          PreparedStatement statement = connection.prepareStatement(sql)) {

        statement.setInt(1, transactionId);
        return statement.executeUpdate() == 1;
      } catch (SQLException ex) {
        ex.printStackTrace();
        return false;
      }
```

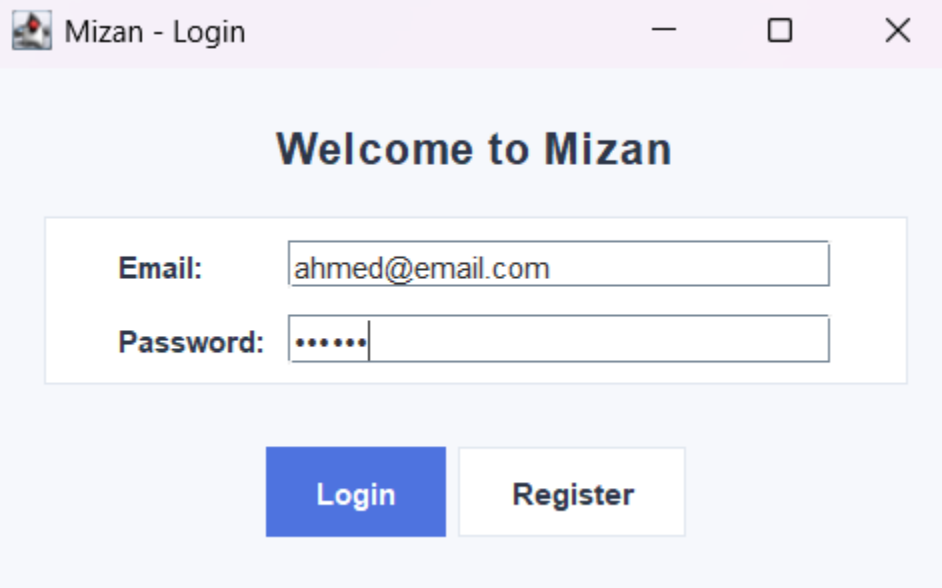
```
    }

    public BigDecimal getMonthlyIncome(int userId, YearMonth yearMonth) {
        String sql = "SELECT COALESCE(SUM(amount), 0) AS total_income "
                + "FROM TRANSACTIONS WHERE user_id = ? AND type = 'Income' "
                + "AND YEAR(date) = ? AND MONTH(date) = ?";
        return sumForMonth(userId, yearMonth, sql);
    }

    public BigDecimal getMonthlyExpense(int userId, YearMonth yearMonth) {
        String sql = "SELECT COALESCE(SUM(amount), 0) AS total_expense "
                + "FROM TRANSACTIONS WHERE user_id = ? AND type = 'Expense' "
                + "AND YEAR(date) = ? AND MONTH(date) = ?";
        return sumForMonth(userId, yearMonth, sql);
    }

    private BigDecimal sumForMonth(int userId, YearMonth yearMonth, String sql) {
        try (Connection connection = getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setInt(1, userId);
            statement.setInt(2, yearMonth.getYear());
            statement.setInt(3, yearMonth.getMonthValue());
            ResultSet rs = statement.executeQuery();
            if (rs.next()) {
                return rs.getBigDecimal(1) != null ? rs.getBigDecimal(1) : BigDecimal.ZERO;
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
        return BigDecimal.ZERO;
    }

    public boolean ensureCategoryExists(int userId, String name, String type) {
        String select = "SELECT category_id FROM CATEGORY WHERE user_id = ? AND name = ?";
        String insert = "INSERT INTO CATEGORY (user_id, name, type) VALUES (?, ?, ?)";
        try (Connection connection = getConnection();
             PreparedStatement selectStatement = connection.prepareStatement(select)) {

            selectStatement.setInt(1, userId);
            selectStatement.setString(2, name);
            ResultSet rs = selectStatement.executeQuery();
            if (rs.next()) {
                return true;
            }

            try (PreparedStatement insertStatement = connection.prepareStatement(insert)) {
                insertStatement.setInt(1, userId);
                insertStatement.setString(2, name);
                insertStatement.setString(3, type);
                return insertStatement.executeUpdate() == 1;
            }
        } catch (SQLException ex) {
```

```
            ex.printStackTrace();
            return false;
        }
    }

    public Optional<Category> getCategoryByName(int userId, String name) {
        String sql = "SELECT category_id, user_id, name, type, icon FROM CATEGORY WHERE user_id = ? AND
name = ?";
        try (Connection connection = getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {

            statement.setInt(1, userId);
            statement.setString(2, name);
            ResultSet rs = statement.executeQuery();
            if (rs.next()) {
                return Optional.of(new Category(
                        rs.getInt("category_id"),
                        rs.getInt("user_id"),
                        rs.getString("name"),
                        rs.getString("type"),
                        rs.getString("icon")
                ));
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
        return Optional.empty();
    }

    public Optional<Category> createCategory(int userId, String name, String type) {
        String sql = "INSERT INTO CATEGORY (user_id, name, type) VALUES (?, ?, ?)";
        try (Connection connection = getConnection();
             PreparedStatement statement = connection.prepareStatement(sql,
PreparedStatement.RETURN_GENERATED_KEYS)) {

            statement.setInt(1, userId);
            statement.setString(2, name);
            statement.setString(3, type);
            int affectedRows = statement.executeUpdate();
            if (affectedRows == 0) {
                return Optional.empty();
            }
            ResultSet keys = statement.getGeneratedKeys();
            if (keys.next()) {
                return Optional.of(new Category(keys.getInt(1), userId, name, type, null));
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
        return Optional.empty();
    }

    public LocalDate getServerDate() {
```

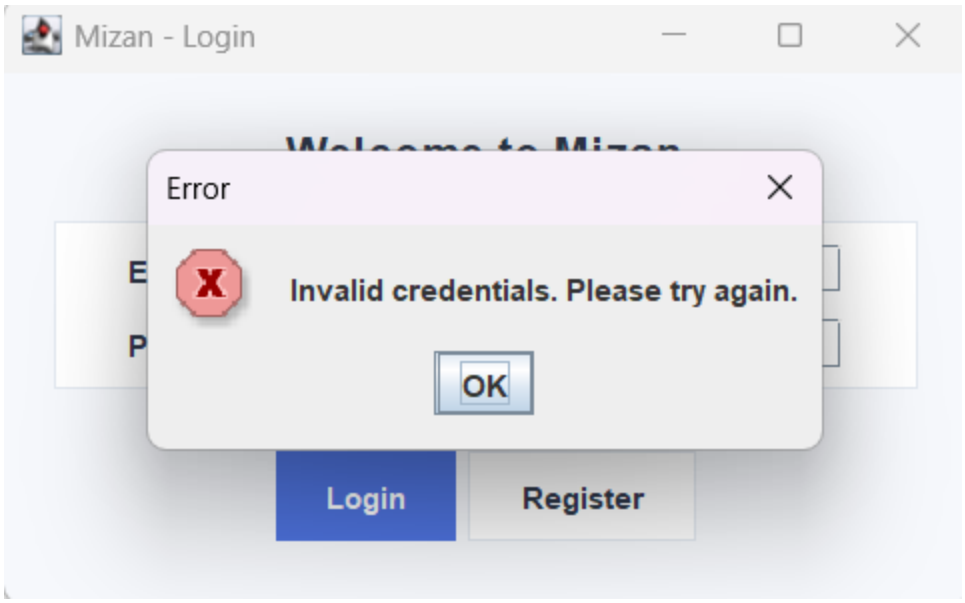
```
        String sql = "SELECT CURDATE() AS current_date";
        try (Connection connection = getConnection();
             PreparedStatement statement = connection.prepareStatement(sql);
             ResultSet rs = statement.executeQuery()) {
            if (rs.next()) {
                Date date = rs.getDate("current_date");
                if (date != null) {
                    return date.toLocalDate();
                }
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
        return LocalDate.now();
    }
}
```

Application Flow and Interfaces

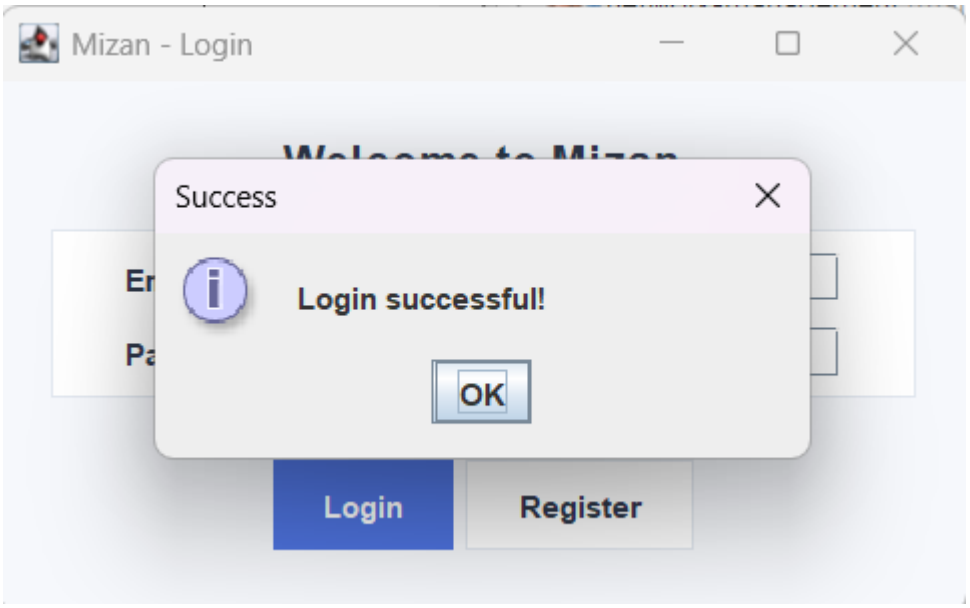
First, the user logs in to his account



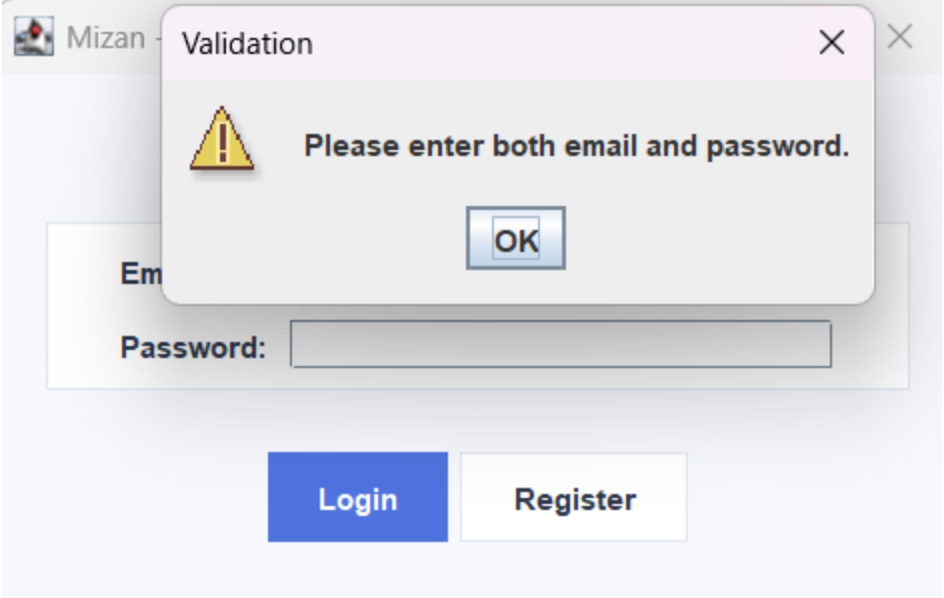
If the email or the password is not correct, the user gets this message:



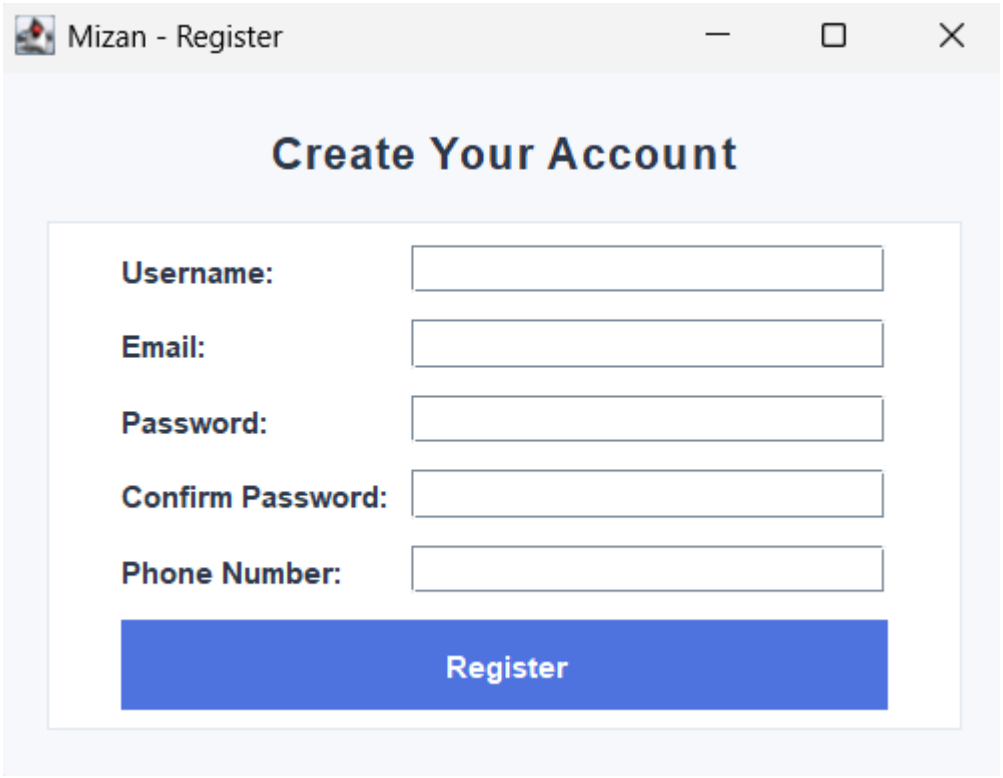
If the email and password are correct, the user gets this message then gets redirected to his dashboard:



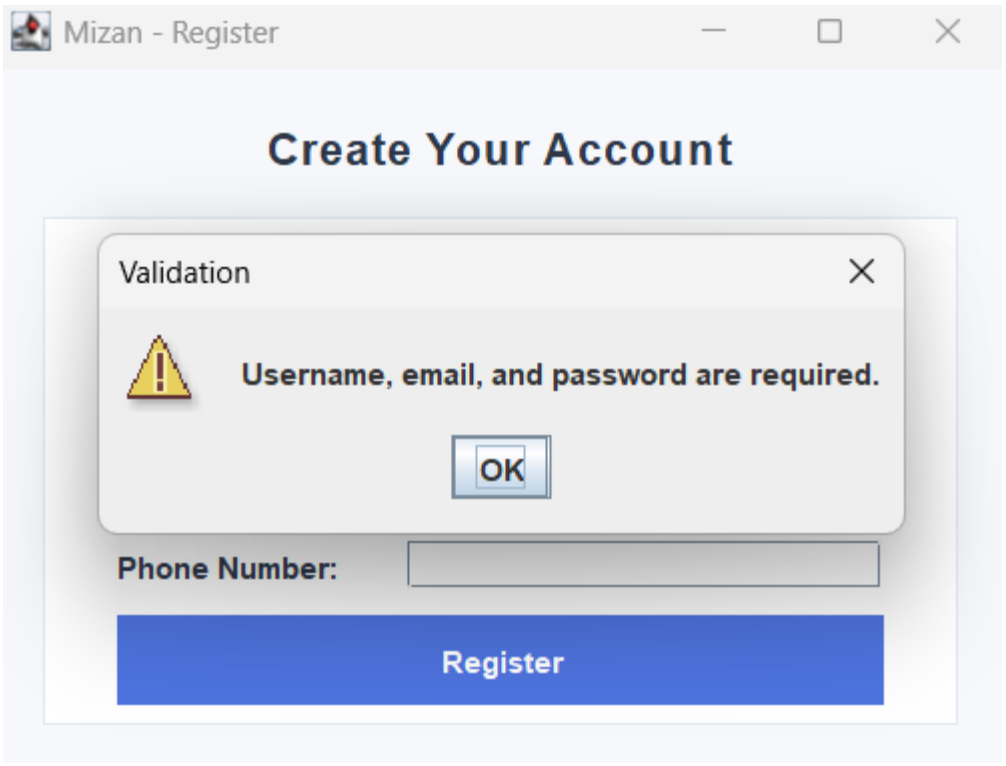
If the user does not enter the email or the password, he will get this message:



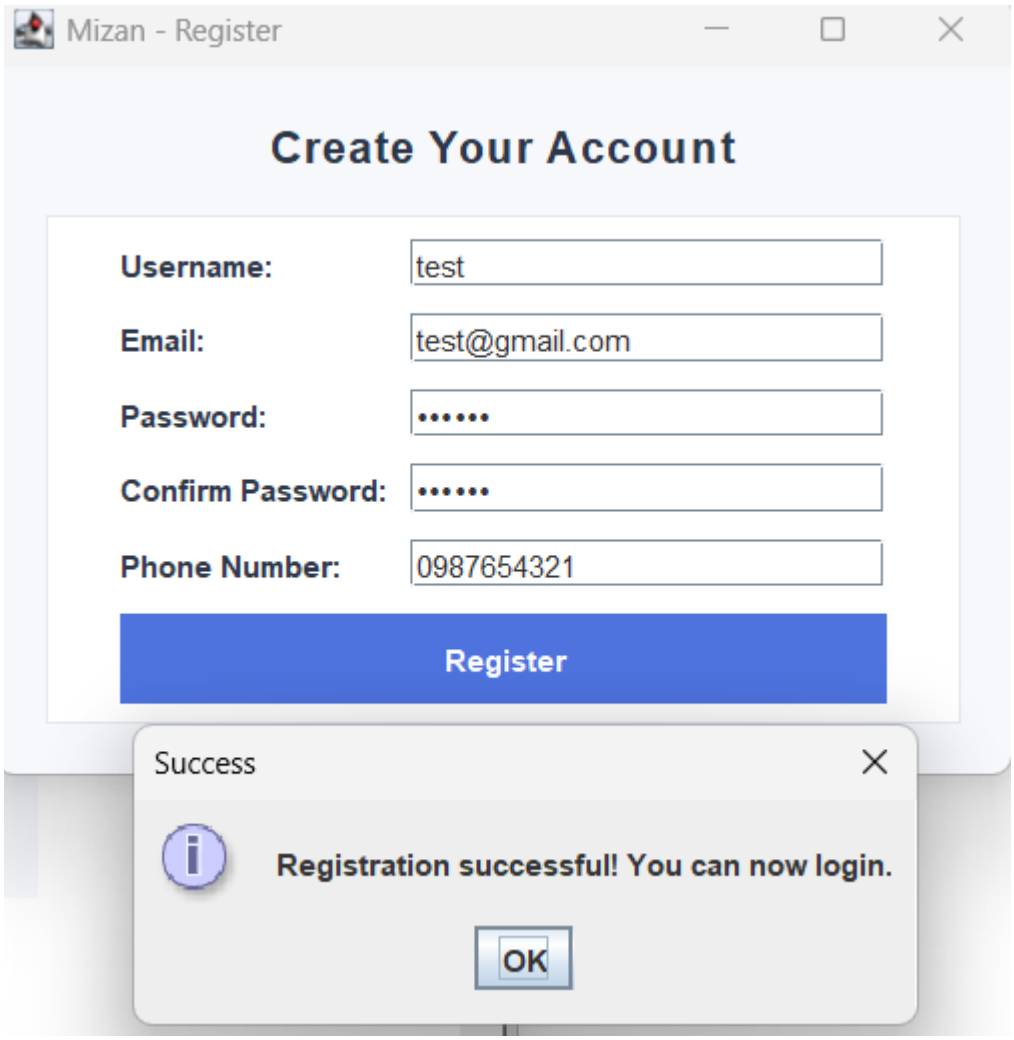
If the user does not have an account, he can create one by registering:



Keeping any required field empty will cause this message to appear:

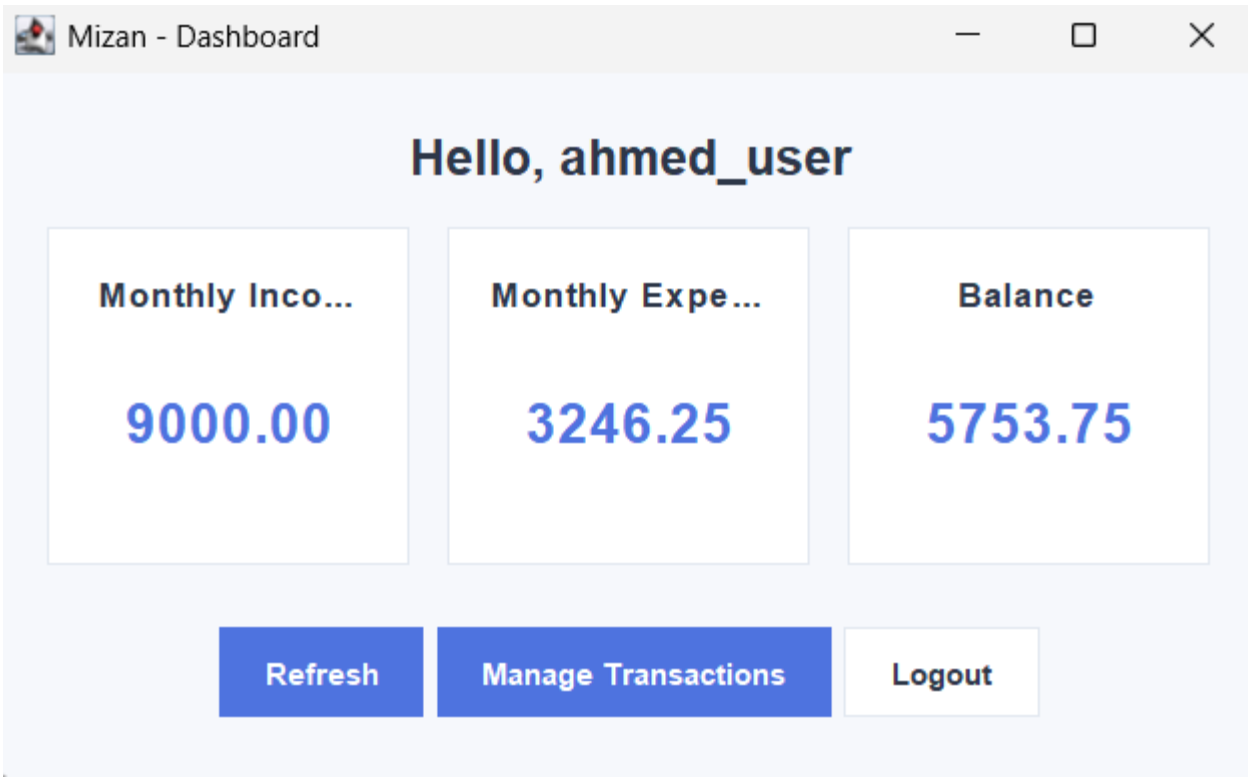


After entering all the information, the user gets the following success message:

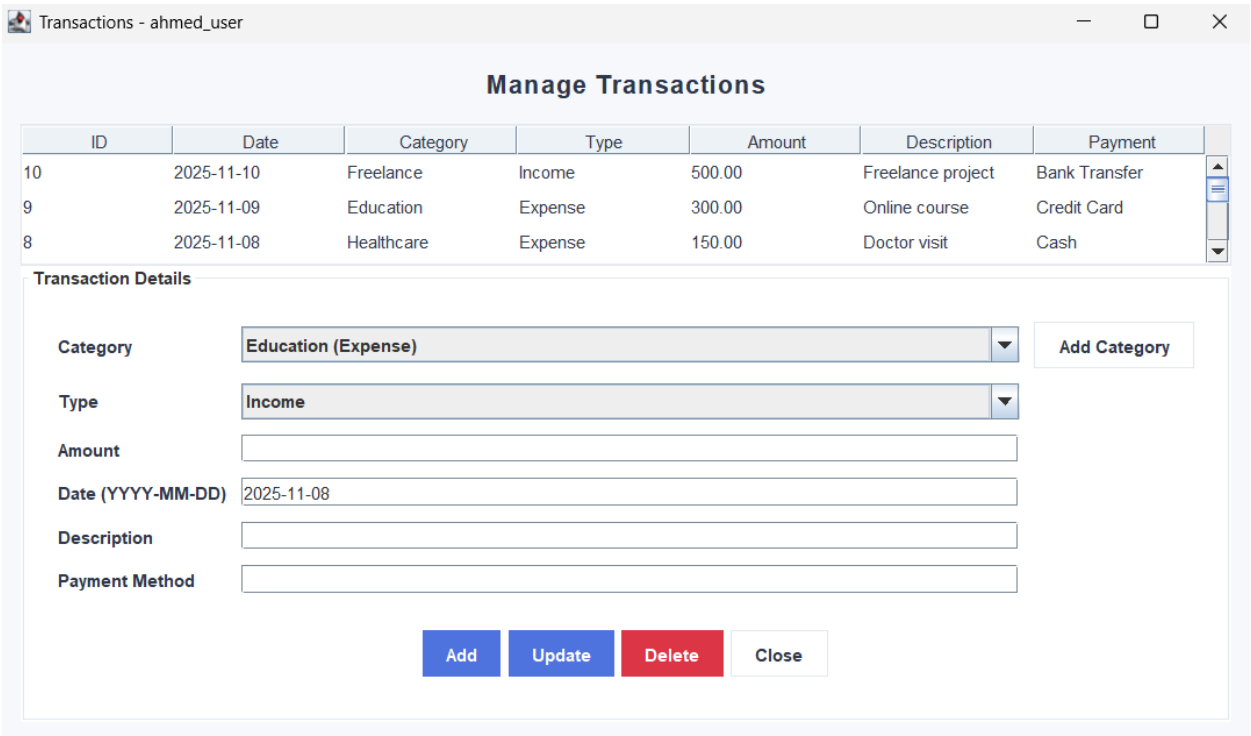


After clicking OK, the user gets redirected to the login page.

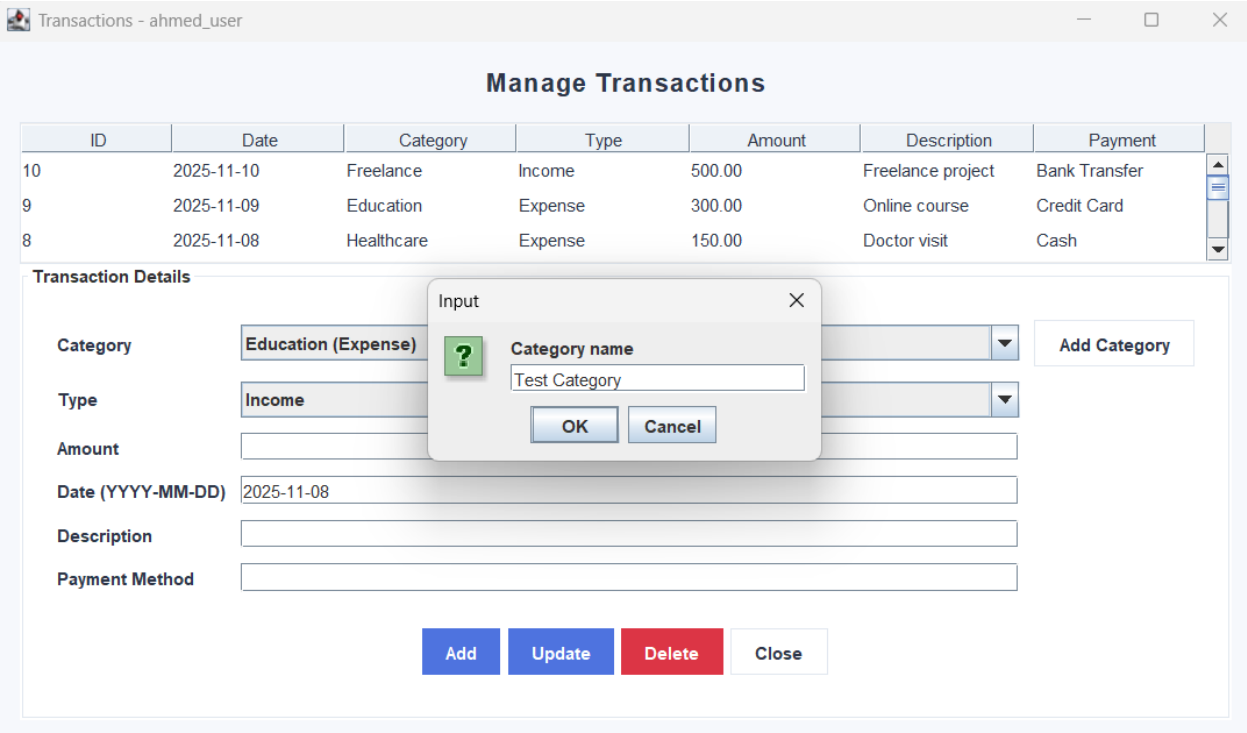
Now when the user is logged in, he can see the dashboard page. It shows the monthly income, the monthly expenses, and the balance. It also has buttons to refresh the dashboard and to manage the transactions. Here is a screenshot for the dashboard:



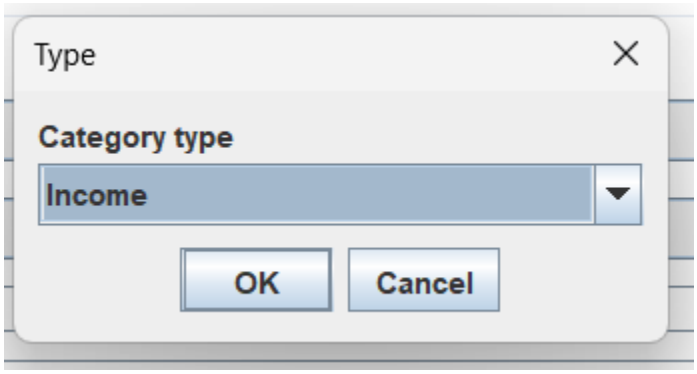
When clicking Manage Transactions, the following page opens:



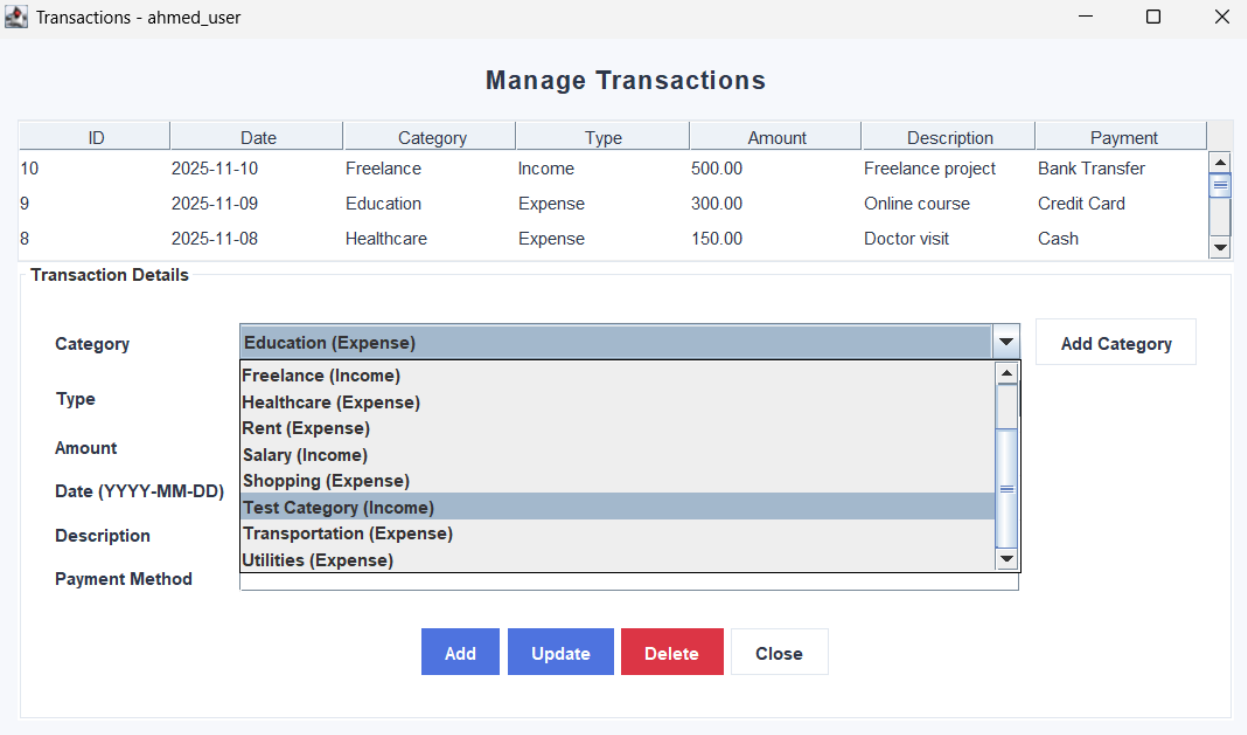
In this page, the user can view the transaction he made and can add new transactions. If he needs to add a new category, he can click Add Category, and a popup will open for him as the following:



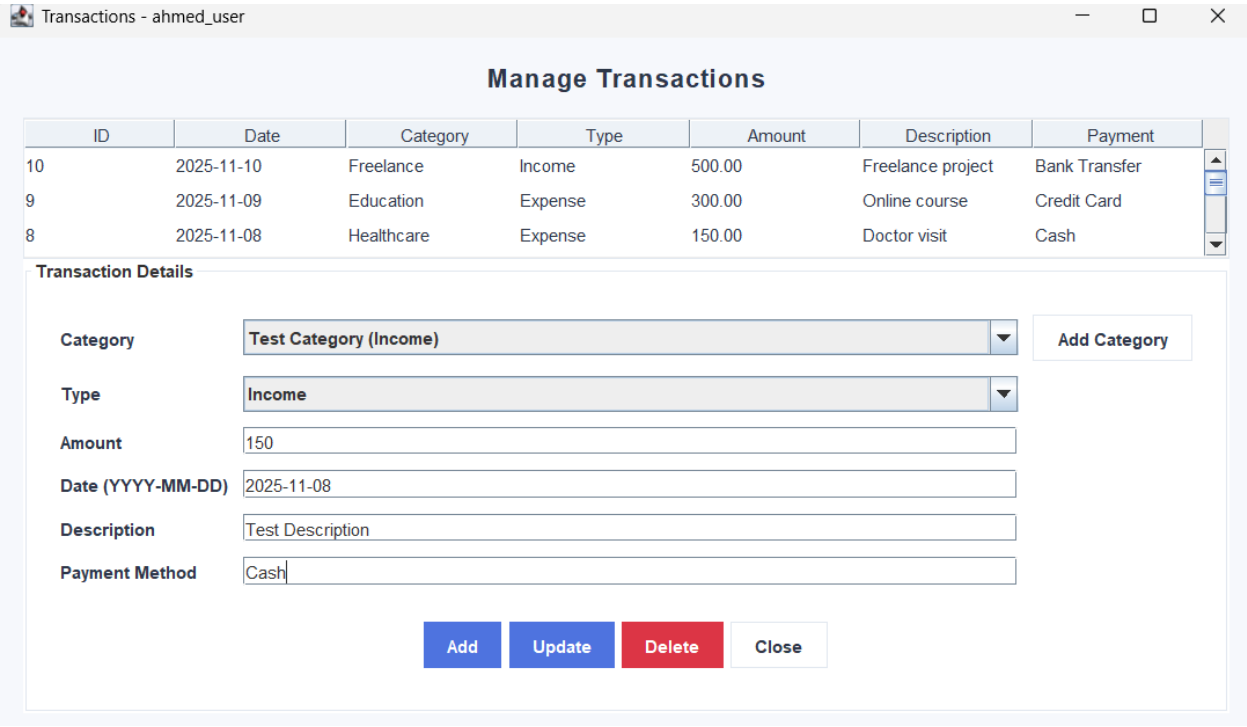
After clicking OK, it will ask the user to enter the type of the new category



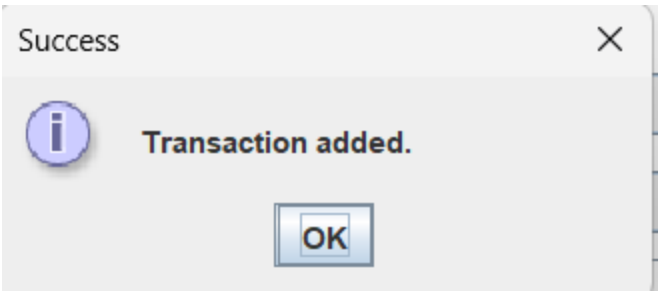
Now we can see it is added:



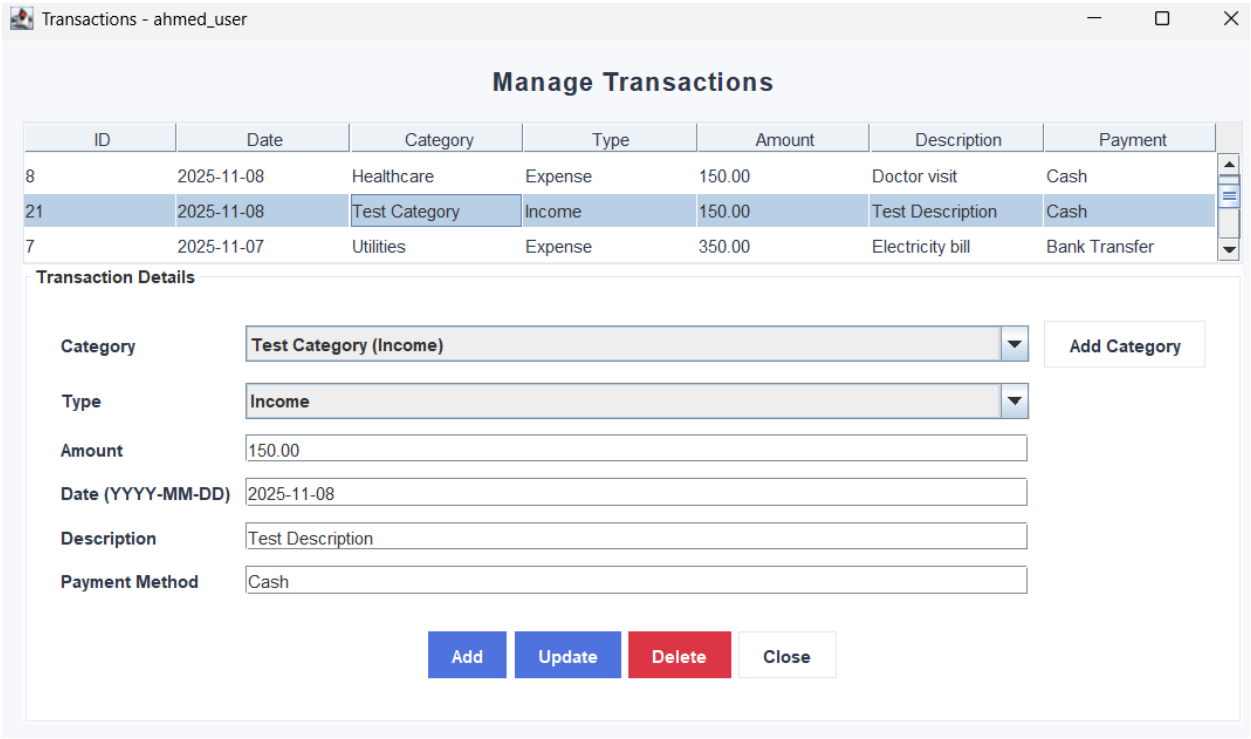
Now if the user fills the info and click on Add as the following:



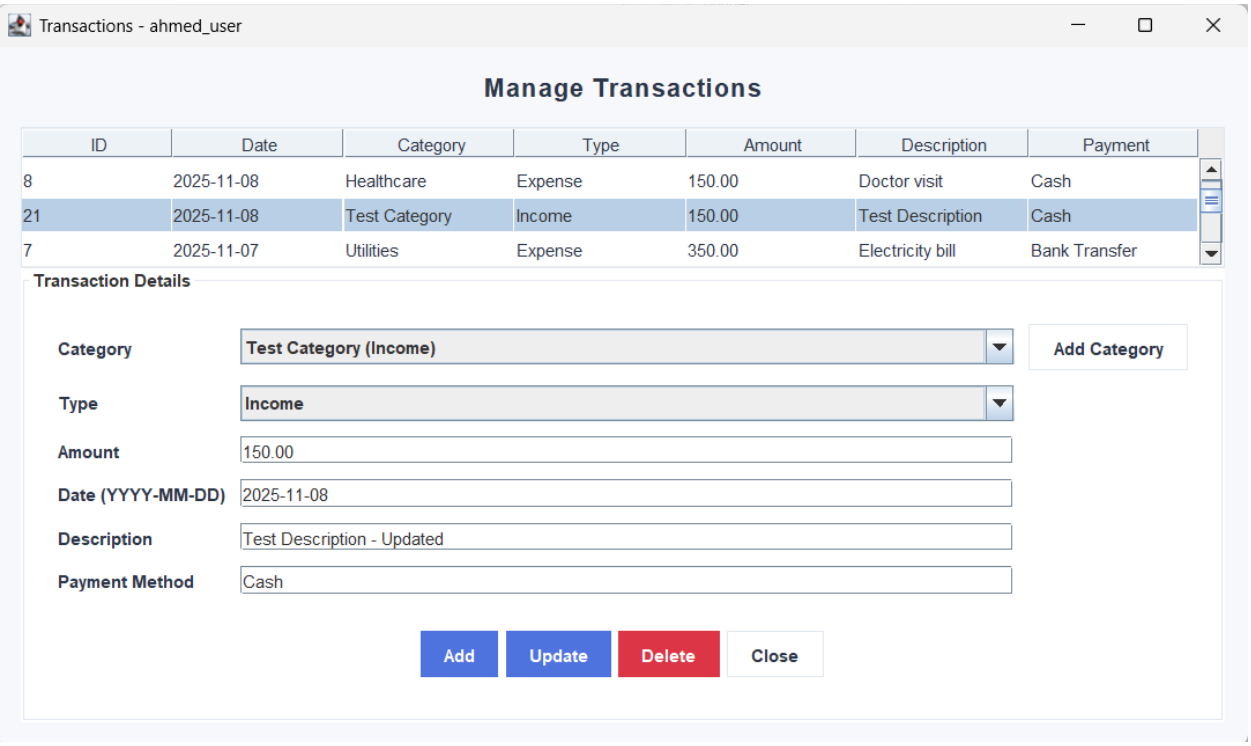
He will get a success message



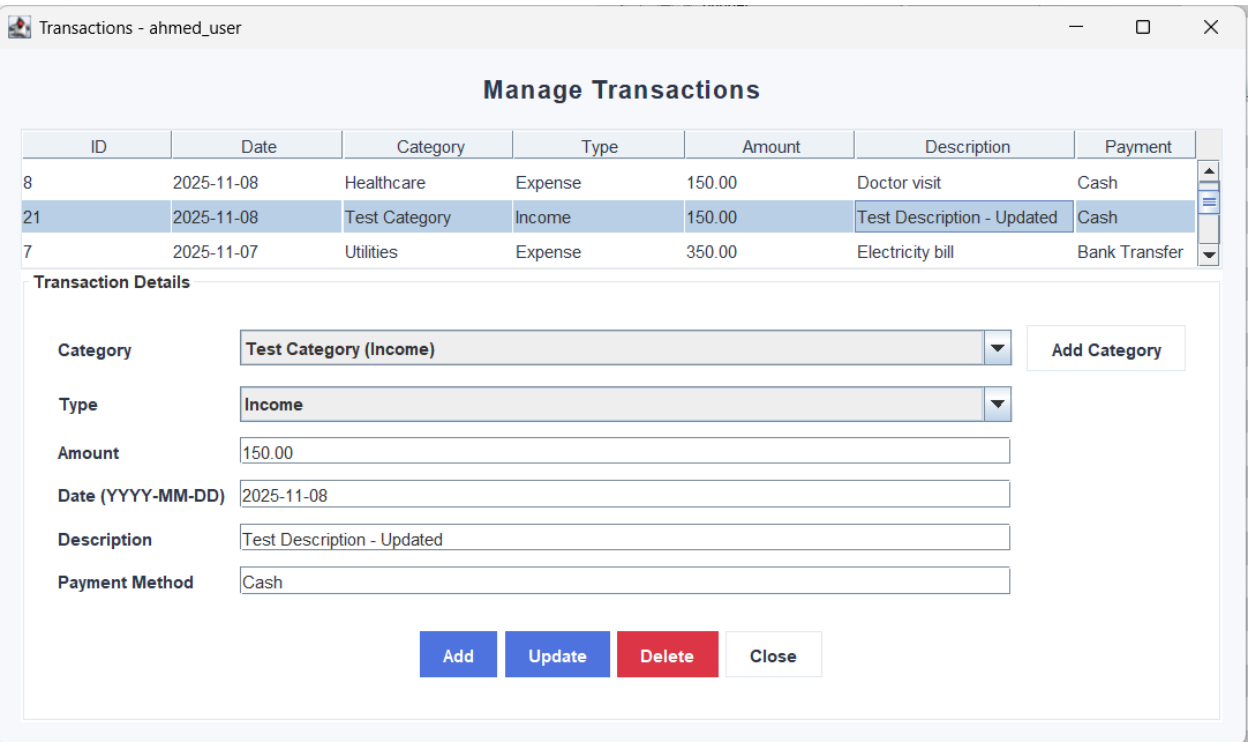
And we can see it in the transactions table



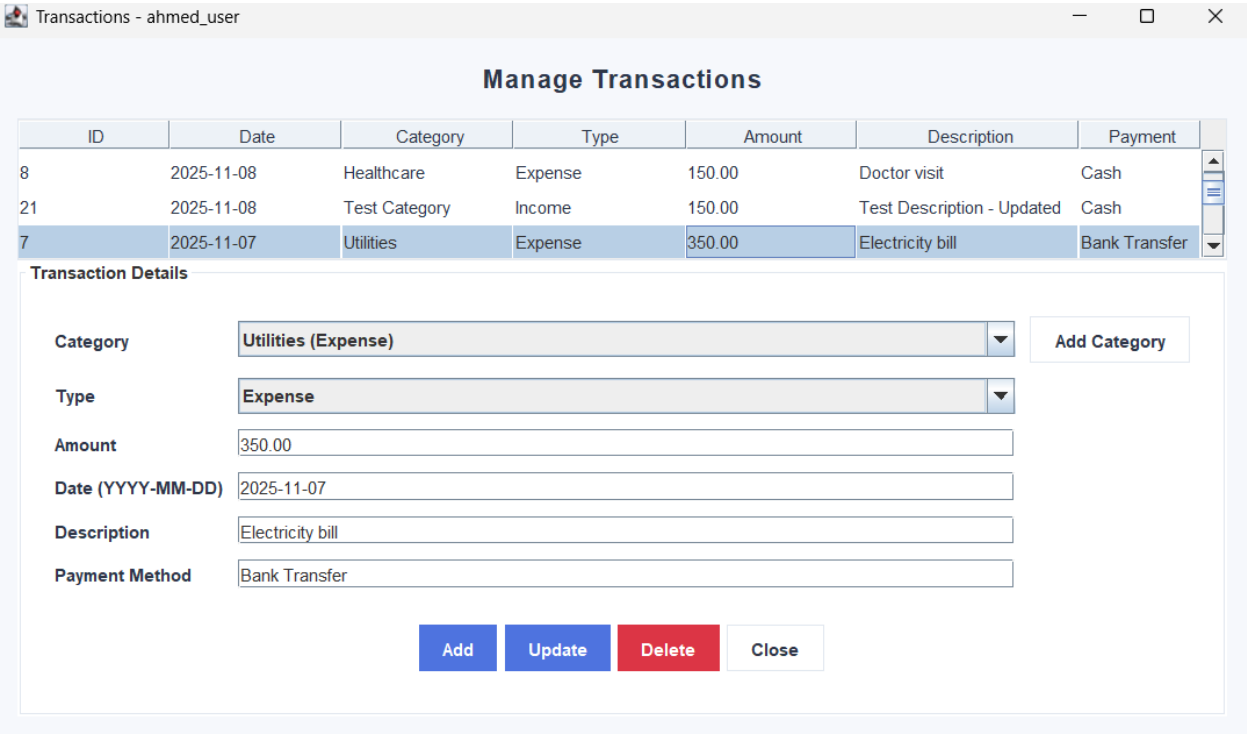
When clicking on a transaction, its information will be filled in the form below to be updated if needed as the following:



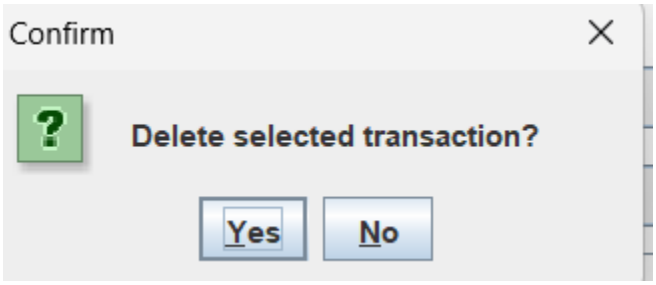
Now we can see that the table is updated



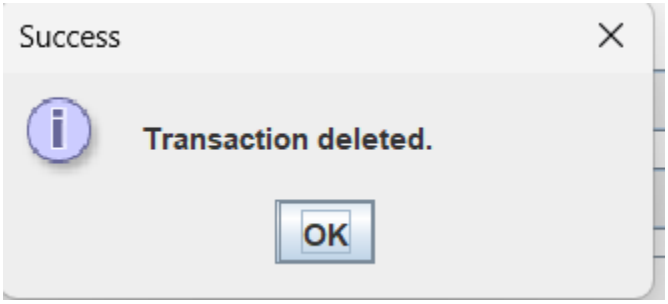
Now we will try to delete this one:



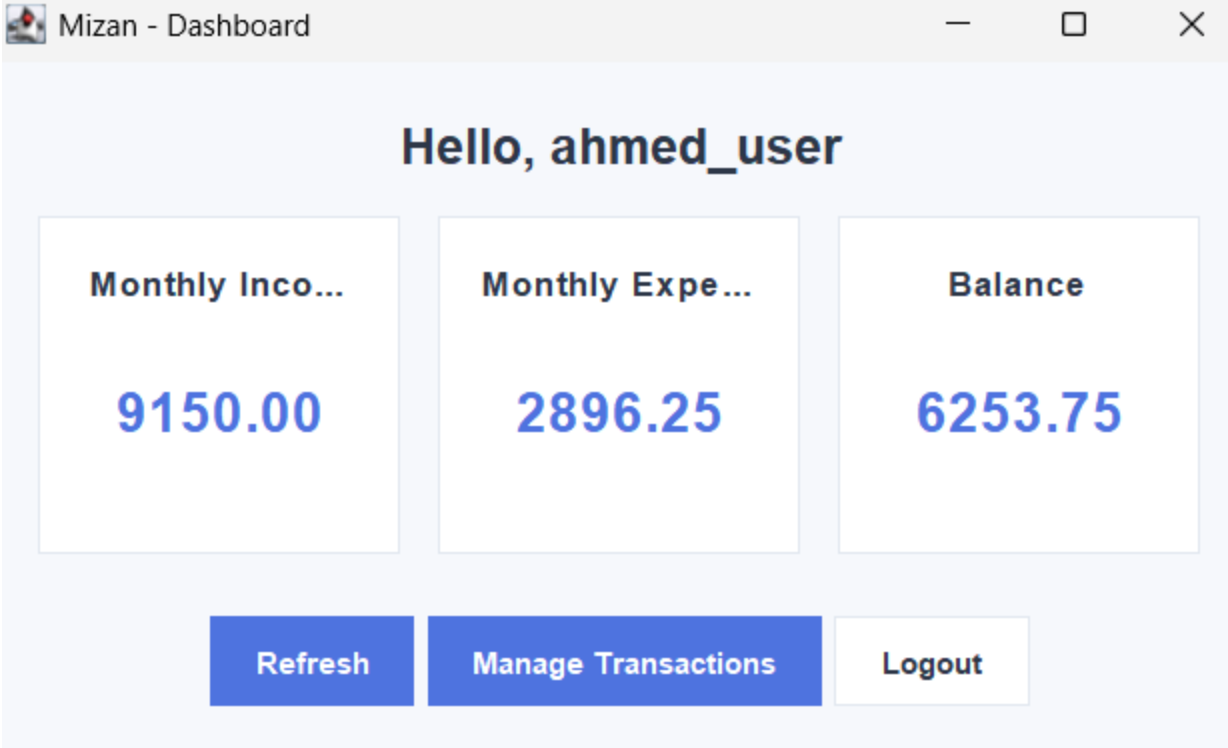
First we get a confirmation message:



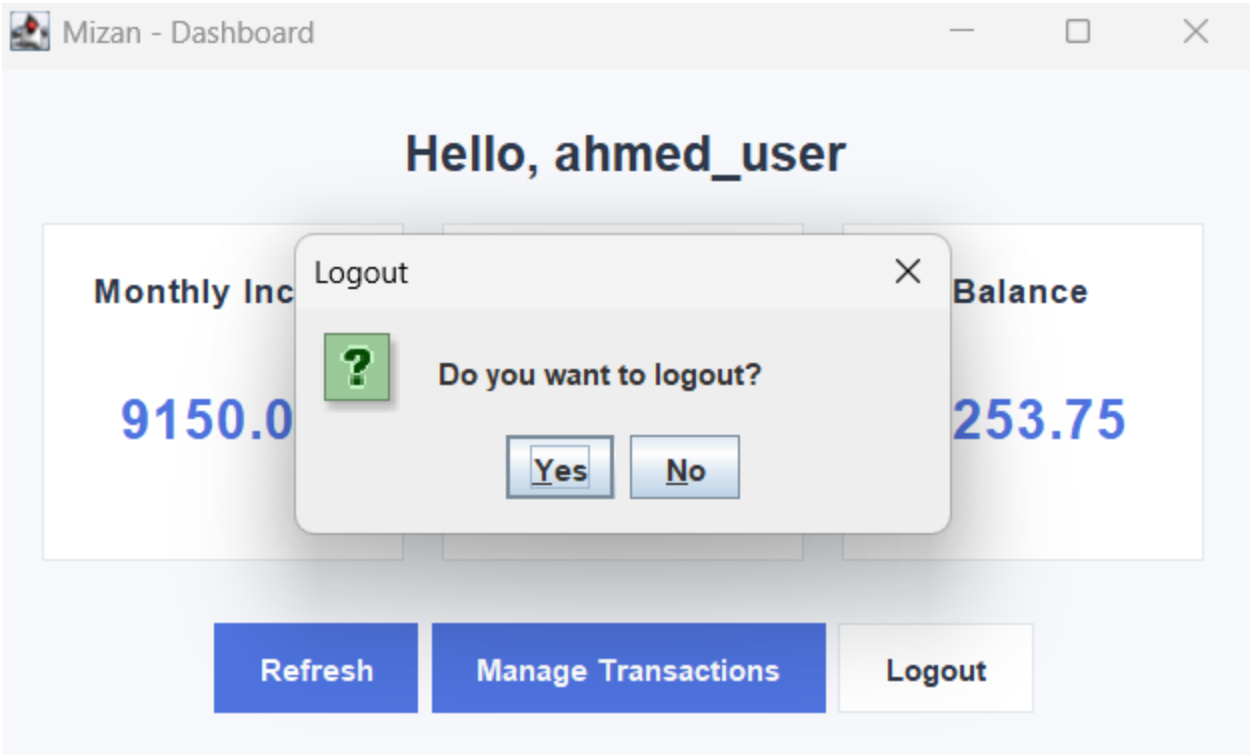
Then we get the success message:



Now in the previous steps we had added a new income, and removed an expense. Therefore, we should notice in the dashboard that the income is now more than before, and the expenses are less, and the balance changed:



Finally, the user can logout and the login page will open again:



Team Members' Contributions

Task	Member
- **Wrote 5 basic queries and 5 advanced one.** - **Handled the database helper main logic and the transaction and category methdos.**	Norah Alfayez
- **Wrote 5 basic queries and 5 advanced one.** - **Built the authentication flow including the login and register.**	Reema Alkheraif
- **Wrote 5 basic queries and 5 advanced one.** - **Created the dashboard view and updating the transactions.**	Mashael Aljarbou
- **Wrote 5 basic queries and 5 advanced one.** - **Created the transaction management page with adding and deleting transactions.**	Jana Altalhi
- **Application Validation & Testing** - **Documentation & Final Review**	All Members